



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Robot air hockey via hierarchical reinforcement learning

TESI DI LAUREA MAGISTRALE IN
COMPUTER SCIENCE AND ENGINEERING - INGEGNERIA IN-
FORMATICA

Author: **Thomas Jean Bernard Bonenfant**

Student ID: 976252

Advisor: Prof. Marcello Restelli

Co-advisors: Amarildo Likmeta, Davide Salaorni

Academic Year: 2023-24

Abstract

Air Hockey is a competitive, dynamic, and complex game. Utilizing robotic systems to play Air Hockey presents various challenges, including physical limitations and constraints that must be respected to ensure safety and prevent damage to the components. This thesis explores the application of Reinforcement Learning techniques in this context. Specifically, since Air Hockey comprises several sub-problems, such as hitting the puck to score a point or defending one's goal, a hierarchical optimization problem is formulated to develop an agent capable of playing entire matches while adhering to physical constraints. This agent is organized into two levels. The lower level comprises several specialized policies, trained using Deep RL and Rule-based RL algorithms, while the higher level selects which specialized policy to use through a parametric state machine, optimized using Rule-Based RL. Additionally, it describes our participation in the Robot Air Hockey Challenge, a competition designed to address the *sim-to-real gap*, the performance and safety drop observed when a real robotic system is controlled by a policy trained in a simulated environment.

Keywords: Robotics, Air Hockey, Rule-based policies, Reinforcement Learning, Deep Reinforcement Learning, Hierarchical Reinforcement Learning, Explainable Artificial Intelligence

Abstract in lingua italiana

Air Hockey è un gioco competitivo, dinamico e complesso. L'impiego di sistemi robotici per giocare ad Air Hockey presenta varie sfide, tra cui limiti fisici e vincoli che devono essere rispettati per garantire la sicurezza ed evitare danni ai componenti. Questa tesi esplora l'applicazione di tecniche di Reinforcement Learning in questo contesto. In particolare, dato che Air Hockey è composto da diversi sottoproblemi, come colpire il disco per segnare un punto o difendere la propria porta, viene formulato un problema di ottimizzazione gerarchica per sviluppare un agente capace di giocare intere partite rispettando i vincoli fisici. Questo agente è organizzato in due livelli. Il livello più basso comprende diverse politiche specializzate, che vengono allenate con algoritmi di Deep RL e Rule-based RL, mentre il livello superiore seleziona quale politica specializzata utilizzare tramite una macchina a stati parametrica, ottimizzata tramite Rule-Based RL. Inoltre, viene descritta la nostra partecipazione alla Robot Air Hockey Challenge, una competizione ideata per affrontare il *sim-to-real gap*, ovvero il calo di performance e sicurezza che si verifica quando un sistema robotico reale è controllato da una politica allenata in un ambiente simulato.

Parole chiave: Robotica, Air Hockey, Politiche a regole, Reinforcement Learning, Deep Reinforcement Learning, Hierarchical Reinforcement Learning, Explainable Artificial Intelligence

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
Introduction	1
1 Preliminaries	5
1.1 Markov Decision Process	5
1.2 Dynamic Programming	10
1.2.1 Policy Evaluation (Prediction)	11
1.2.2 Policy Improvement	11
1.3 Reinforcement Learning	13
1.3.1 Temporal-Difference Learning	13
1.3.2 Reinforcement Learning Taxonomy	17
1.3.3 Deep Reinforcement Learning	19
2 Related Works	23
2.1 Controller for the Air Hockey task	23
2.2 Reinforcement Learning in Robotics	25
2.3 ATACOM	28
3 Robot Air Hockey Challenge	31
3.0.1 Air Hockey Game	32
3.0.2 Motivation	32
3.1 Challenge organization	33
3.1.1 Warm-Up Stage	33
3.1.2 Qualifying Stage	33
3.1.3 Tournament Stage	34

3.2	Framework	35
3.2.1	Environments	36
3.2.2	Robot models	38
3.2.3	Constraints	39
3.2.4	Evaluation Metrics	40
3.2.5	Simulator	41
4	Methodology	43
4.1	Air hockey as an MDP	43
4.1.1	State space	43
4.1.2	Action space	44
4.1.3	Reward function	44
4.2	SAC Agent	47
4.3	RL Optimized Rule-based Agent	49
4.4	Hierarchical Agent	50
4.5	Noise Filtering	51
5	Experimental results	53
5.1	Challenge Results	53
5.1.1	Warm up phase	53
5.1.2	Qualifying phase	53
5.1.3	Tournament	54
5.2	Improving the challenge's solution	55
5.2.1	Improving the Hit Task	55
5.2.2	Improving the Switcher	56
6	Conclusions and future developments	59
6.1	Future works	59
	Bibliography	61
	List of Figures	65
	List of Tables	67
	List of Acronyms	69

Introduction

Sequential decision making involves making a series of decisions over time, with each decision influencing future states and subsequent decisions. Actions are dynamically selected based on current and past information to achieve a long-term objective. Several techniques are available to achieve this long-term objective, and the choice of technique typically depends on the level of knowledge about the environment.

When perfect knowledge of the environment is available, a solution can be found via dynamic programming or linear programming techniques. However, in most cases, a perfect model of the environment is not available, necessitating the use of the reinforcement learning framework.

Reinforcement learning enables learning from trial and error by interacting with the environment, which provides states and rewards. However, applying this trial-and-error approach can be problematic in some applications. For example, in real-world robotic control, this approach can lead to safety issues and unwanted damage to the robotic system and its environment. Therefore, reinforcement learning is often first applied in a simulated environment before being deployed in the real world.

This transfer from simulation to the real world often suffers from performance drops and still presents safety and damage issues because the simulation model may not be accurate or may not account for noise and other real-world problems. This discrepancy between simulation and the real world is known as the reality gap.

Motivation

Closing the reality gap is a crucial research direction in achieving true embodied intelligence. To attain this goal, multiple factors must be considered: in real-world scenarios, we encounter uncertainties stemming from sensor noise, modeling errors, and delays. Additionally, agents deployed on real robotic systems must operate without damaging their components, the environment, or causing harm to people.

The game of air hockey presents a complex and dynamic challenge where a robotic player

must be highly reactive, adhere to constraints, and exhibit long-term planning capabilities. This scenario compels engineers to push the boundaries of robotic capabilities while ensuring safety.

With these challenges in mind, the Robot Air Hockey competition was organized. This competition serves as a platform for researchers in the field of robot learning to engage with a realistic robotic task. Teams participating in the competition design and build their air hockey agents, competing against each other in various subtasks (in simulation) and ultimately in full games (both in simulation and in the real world).

To be successful, an agent must solve multiple sub-problems, such as scoring goals against an opponent, effectively defending its own goal, and devising winning strategies. The inherent sub-problem composition of air hockey inspires this work, where a hierarchical approach to the game is investigated. Reinforcement learning techniques are applied to each sub-problem that contributes to a complete air hockey match. These specialized agents are then integrated into a unified high-level agent that continuously selects the optimal low-level agent to deploy during the match.

Goal

The goal of this work is to develop an agent capable of playing entire air hockey games while adhering to all operational constraints. Specifically, the agent must control a general-purpose manipulator, achieving the best possible performance in terms both of offense and defense. It is crucial that the agent maintains these performance levels without violating any constraints, ensuring that no drop in performance or damage concerns arise when deployed on a real robot. The agent must:

- react quickly and accurately to the puck's movements.
- Maintain strategic positioning and make decisions that balance offensive and defensive play.
- Operate within safety margins, ensuring that the manipulator does not exceed its physical limits or damage constraints.

Thesis Structure

The thesis is structured as follows:

1. **Preliminary Background:** This section introduces the main theories of Rein-

forcement Learning and presents state-of-the-art Deep Reinforcement Learning algorithms used for continuous control, as well as a brief overview of hierarchical reinforcement learning frameworks.

2. **Related Works:** This chapter reviews existing literature on trajectory optimization algorithms used for robotics control and describes the main challenges in applying reinforcement learning techniques to robotics.
3. **Air Hockey Competition:** This chapter describes the competition, its rules, and the provided framework.
4. **Methodology:** This chapter details the design and implementation of the agents used during the competition, including the hierarchical approach and reinforcement learning techniques applied.
5. **Experimental Results:** This chapter presents the results obtained during the competition and discusses the improvements made post-competition.
6. **Conclusions and future developments**

1 | Preliminaries

In this chapter, we introduce the fundamental theories necessary for understanding and solving sequential decision problems. The chapter is organized into three main sections to provide a comprehensive overview of the foundational concepts.

The first section delves into the Markov Decision Process (MDP) framework, which is essential for modeling sequential decision problems. This section covers the formal definition of MDPs and discusses the theoretical solution for these problems, highlighting the limitations of this closed-form solution in practical scenarios.

The second section explores Dynamic Programming (DP), an iterative approach to solving MDPs. Here, we examine the key principles and algorithms of dynamic programming, which offer a more feasible solution method compared to the theoretical closed-form solution.

The final section introduces Reinforcement Learning (RL), a methodology that builds upon the results of dynamic programming. Reinforcement learning provides a framework for agents to learn optimal behaviors through interactions with their environment, making it a powerful tool for solving complex sequential decision problems.

The majority of the following information is derived from [16].

1.1. Markov Decision Process

In a Sequential Decision problem an agent interacts with an environment through actions and receives a feedback signal from it. The environment is usually modeled as a Markov Decision Process.

Definition 1.1.1. *A Finite Markov Decision Process (MDP) is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma, \mu)$ specified by:*

- *a finite set $\mathcal{S}$ of possible states*

- a finite set $\mathcal{A}$ of actions
- a transition probability function $p : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$
- a reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$
- a discount factor $\gamma \in (0, 1)$
- a probability distribution over the initial states $\mu : \mathcal{S} \rightarrow [0, 1]$

The agent interacts with it at discrete time steps $t = 0, 1, 2, 3, \dots$. At each time step the agent receives some representation of the environment's *state*, $S_t \in \mathcal{S}$, and on that basis selects an *action*, $A_t \in \mathcal{A}$. After performing an action, the agent receives a numerical *reward*, $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$, and finds itself in a new state, S_{t+1} . The MDP and agent together thereby give rise to a sequence or *trajectory* that begins like this:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

The transition probability depends only on the previous state and action thanks to the Markov assumption:

$$p(s'|s, a) \doteq \mathbb{P}(S_t = s' | S_{t-1} = s, A_{t-1} = a)$$

The reward function is the expected reward the state action pair:

$$r(s, a) \doteq \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a]$$

The agent chooses actions according to a policy. A policy π is a distribution over actions given the state:

$$\pi(a|s) = \mathbb{P}(a|s)$$

Therefore the policy can be stochastic or deterministic and because it considers only the current state it is also Markovian. The goal of the agent is to learn a policy π that maximizes the expected return, where the return is defined as the cumulative discounted reward:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Given a policy π , it is possible to define the utility of each state through a value function.

Definition 1.1.2. The state value function $v^\pi : \mathcal{S} \rightarrow \mathbb{R}$ of an MDP is the expected return starting from state s and then following policy π

$$v^\pi(s) \doteq \mathbb{E}_\pi[G_t | S_t = s], \quad \forall s \in \mathcal{S}$$

$\mathbb{E}_\pi[\cdot]$ denotes the expected value of a random variable given that the agent follows policy π . For control purposes, rather than the value of each state we consider the value of each action in each state.

Definition 1.1.3. The action-value function $q^\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ of an MDP is the expected return starting from state s , taking action a and then following policy π

$$q^\pi(s, a) \doteq \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

A fundamental property of the value functions is that they satisfy the following recursive relationships, called *Bellman expectation equations*:

$$v^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v^\pi(s') \right), \quad \forall s \in \mathcal{S} \quad (1.1)$$

$$q^\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \sum_{a' \in \mathcal{A}} \pi(a'|s') q^\pi(s', a'), \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A} \quad (1.2)$$

Definition 1.1.4. The Markov Reward Process induced by a policy π on a MDP is the tuple $(\mathcal{S}, p^\pi, r^\pi)$ specified by:

- the same set $\mathcal{S}$ from the MDP
- the transition probability function $p^\pi : \mathcal{S} \times \mathcal{S} \rightarrow [0, 1]$ obtained from following policy π
- the reward function $r^\pi : \mathcal{S} \rightarrow \mathbb{R}$ obtained from following policy π

$$p^\pi(s'|s) = \mathbb{P}_\pi(S_t = s' | S_{t-1} = s) = \sum_{a \in \mathcal{A}} \pi(a|s) p(s'|s, a)$$

$$r^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) r(s, a)$$

The bellman expectation equation for v^π can be written in matrix form using the induced MRP and a closed form solution can be computed:

$$V^\pi = R^\pi + \gamma P^\pi V^\pi$$

$$V^\pi = (I - P^\pi)^{-1} R^\pi$$

where $V^\pi \in \mathbb{R}^{|\mathcal{S}|}$ is the matrix equivalent of v^π .

Because $\lambda < 1$, the matrix $(I - P^\pi)$ is always invertible and thus a solution is guaranteed to exist.

Although closed form solutions exist to compute the value function of a tabular MDP, they are very computationally expensive due to the matrix inversion. Alternatively, iterative solutions exist thanks to the definition of the Bellman operators.

Definition 1.1.5. *The Bellman operator for V^π is defined as $T^\pi : \mathbb{R}^{|\mathcal{S}|} \rightarrow \mathbb{R}^{|\mathcal{S}|}$*

$$(T^\pi v^\pi)(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v^\pi(s') \right), \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}$$

Using the Bellman operator for v^π , the Bellman expectation equation can be compactly written as:

$$T^\pi V^\pi = V^\pi,$$

i.e. the value function is a fixed point of the Bellman expectation operator.

Definition 1.1.6. *The Bellman operator for q^π is defined as $T^\pi : \mathbb{R}^{|\mathcal{S} \times \mathcal{A}|} \rightarrow \mathbb{R}^{|\mathcal{S} \times \mathcal{A}|}$*

$$(T^\pi q^\pi)(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \sum_{a' \in \mathcal{A}} \pi(a'|s') q^\pi(s, a'), \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}$$

Using the Bellman operator for q^π , the Bellman expectation equations can be compactly written as:

$$T^\pi Q^\pi = Q^\pi,$$

i.e. Q^π is a fixed point of the Bellman Expectation operator.

Definition 1.1.7. *The optimal state-value function $v^*(s)$ is the maximum state-value function over all policies*

$$v^*(s) = \max_{\pi} v^\pi(s), \quad \forall s \in \mathcal{S}$$

Definition 1.1.8. *The optimal action-value function $q^*(s)$ is the maximum action-value*

function over all policies

$$q^*(s, a) = \max_{\pi} q^{\pi}(s, a), \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}$$

Also the optimal value functions satisfy a recursive relationship, called *Bellman Optimality Equations*

$$\begin{aligned} v^*(s) &= \max_{a \in \mathcal{A}} q^*(s, a) \\ &= \max_{a \in \mathcal{A}} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v^*(s') \right), \quad \forall s \in \mathcal{S} \end{aligned} \quad (1.3)$$

$$\begin{aligned} q^*(s, a) &= r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v^*(s') \\ &= r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \max_{a' \in \mathcal{A}} q^*(s', a'), \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A} \end{aligned} \quad (1.4)$$

Value functions define a partial ordering over policies

$$\pi > \pi' \text{ if } v^{\pi}(s) > v^{\pi'}(s), \quad \forall s \in \mathcal{S}$$

Theorem 1.1. *For any Markov Decision Process*

1. *There exists an optimal policy π^* that is better than or equal to all other policies*
 $\pi^* \geq \pi, \quad \forall \pi$
2. *All optimal policies achieve the optimal value function $v^{\pi^*}(s) = v^*(s)$*
3. *All optimal policies achieve the optimal state-value function $q^{\pi^*}(s, a) = q^*(s, a)$*
4. *There is always a deterministic optimal policy for an MDP*

A deterministic optimal policy can be found by maximizing over $q^*(s, a)$ (greedy policy)

$$\pi^*(s, a) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} q^*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

Definition 1.1.9. The Bellman optimality operator for v^* is defined as $T^* : \mathbb{R}^{|\mathcal{S}|} \rightarrow \mathbb{R}^{|\mathcal{S}|}$

$$(T^*v^*)(s) = \max_{a \in \mathcal{A}} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v^*(s') \right)$$

Definition 1.1.10. The Bellman optimality operator for q^* is defined as $T^* : \mathbb{R}^{|\mathcal{S} \times \mathcal{A}|} \rightarrow \mathbb{R}^{|\mathcal{S} \times \mathcal{A}|}$

$$(T^*q^*)(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \max_{a' \in \mathcal{A}} q^*(s', a') \forall (s, a) \in \mathcal{S} \times \mathcal{A}$$

The Bellman operators exhibit the following properties:

- **Monotonicity:** if $f_1 \leq f_2$ component-wise

$$T^\pi f_1 \leq T^\pi f_2 \quad , \quad T^* f_1 \leq T^* f_2$$

- **Max-Norm Contraction:** for two vectors f_1 and f_2

$$\|T^\pi f_1 - T^\pi f_2\|_\infty \leq \gamma \|f_1 - f_2\|_\infty$$

$$\|T^* f_1 - T^* f_2\|_\infty \leq \gamma \|f_1 - f_2\|_\infty$$

- v^π is the **unique fixed point** of T^π
- v^* is the **unique fixed point** of T^*
- For any vector $f \in \mathbb{R}^{|\mathcal{S}|}$ and any policy π , we have

$$\lim_{k \rightarrow \infty} (T^\pi)^k f = v^\pi \quad , \quad \lim_{k \rightarrow \infty} (T^*)^k f = v^*$$

Differently from (1.1), the Bellman optimality equation is non linear due to the presence of a max operation and no closed form solution exists for the general case.

The bellman optimality equation of an MDP is solved by different iterative methods like Dynamic programming, Linear programming and Reinforcement Learning.

1.2. Dynamic Programming

The term dynamic programming (DP) refers to a collection of algorithms that can be used to compute optimal policies given a perfect model of the environment as a Markov Decision Process. Classical DP algorithms are of limited utility in reinforcement learning both because of their assumption of a perfect model and because of their computational

expense

1.2.1. Policy Evaluation (Prediction)

In Dynamic Programming, policy evaluation is done iteratively using the bellman equation for $v^\pi(s)$ (1.1) as an update rule. This update rule is called Bellman Expectation Backup and applying it to each state is called a *sweep*.

$$\begin{aligned} v_{k+1}(s) &\doteq \mathbb{E}_\pi[R_{t+1} + \gamma v_k^{S_{t+1}} | S_t = s] \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_k(s')] \end{aligned}$$

Algorithm 1.1 Iterative Policy Evaluation

- 1: **Input:** A Markov Decision Process (MDP) defined by (S, A, P, R, γ) and a policy π
 - 2: **Output:** State-value function V for policy π
 - 3: **Initialize** $V(s) \leftarrow 0$ for all $s \in S$
 - 4: **Initialize** $\theta \leftarrow$ a small positive number (convergence threshold)
 - 5: **repeat**
 - 6: $\Delta \leftarrow 0$
 - 7: **for** each $s \in S$ **do**
 - 8: $v \leftarrow V(s)$
 - 9: $V(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a|s) [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V(s')]$
 - 10: $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 - 11: **end for**
 - 12: **until** $\Delta < \theta$
 - 13: **return** V
-

1.2.2. Policy Improvement

Considering a deterministic policy π we can improve it by acting greedily:

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q^\pi(s, a)$$

Theorem 1.2. *Let π and π' be any pair of deterministic policies such that*

$$q^\pi(s, \pi'(s)) \geq v^\pi(s) \quad , \quad \forall s \in \mathcal{S}$$

Then the policy π' must be as good as, or better than π

$$v^{\pi'}(s) \geq v^{\pi}(s) \quad , \quad \forall s \in \mathcal{S} \quad (1.5)$$

If the improvement stops ($v^{\pi'}(s) = v^{\pi}(s) \forall s \in \mathcal{S}$) then the Bellman Optimality Equation (1.3) holds and $\pi' = \pi$ is an optimal policy π^* .

The policy iteration algorithm 1.2 alternates between policy evaluation and policy improvements until the policy is stable and thus optimal.

Algorithm 1.2 Policy Iteration

```

1: Input: A Markov Decision Process (MDP) defined by  $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$  and a policy  $\pi$ 
2: Output: Optimal policy  $\pi^*$  and Optimal state-value function  $V^*(s)$ 
3: Initialize  $V(s) \in \mathbb{R}$  and  $\pi(s) \in \mathcal{A}(s)$  arbitrarily for all  $s \in \mathcal{S}$ 
4: Initialize  $\theta \leftarrow$  a small positive number (convergence threshold)
5: repeat
6:    $\Delta \leftarrow 0$ 
7:   for each  $s \in \mathcal{S}$  do
8:      $v \leftarrow V(s)$ 
9:      $V(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a|s)[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a)V(s')]$ 
10:     $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
11:   end for
12: until  $\Delta < \theta$ 
13: policy-stable  $\leftarrow$  true
14: for each  $s \in \mathcal{S}$  do
15:   old-action  $\leftarrow \pi(s)$ 
16:    $\pi(s) \leftarrow \arg \max_{a \in \mathcal{A}(s)} [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) * V(s)]$ 
17:   if old-action  $\neq \pi(s)$  then
18:     policy-stable  $\leftarrow$  false
19:   end if
20: end for
21: if policy-stable then
22:   return  $V \approx v^*$  and  $\pi \approx \pi^*$ 
23: else
24:   go to 5
25: end if

```

A small modification to the policy iteration algorithms lead to the Value iteration algo-

rithm 1.3 in which the evaluation is stopped after only one sweep and followed by the policy improvement. It basically implements the bellman optimality equation (1.3) as an update rule:

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_k(s) \right)$$

Once $v_k \approx v^*$ we can build a greedy policy π^*

Algorithm 1.3 Value Iteration

```

1: Initialize  $V(s) = 0$  for all states  $s \in S$ 
2: repeat
3:    $\Delta \leftarrow 0$ 
4:   for  $s \in S$  do
5:      $v \leftarrow V(s)$ 
6:      $V(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$ 
7:      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8:   end for
9: until  $\Delta < \theta$ 
10: Output:  $V$  and greedy policy  $\pi(s) = \arg \max_{a \in A(s)} (r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V(s'))$ 

```

1.3. Reinforcement Learning

In section 1.2 a perfect model of the environment was used in order to compute the backup updates. Model-Free Reinforcement Learning aims to solve the prediction and control problem only through experience borrowing ideas from Dynamic Programming.

1.3.1. Temporal-Difference Learning

Temporal Difference Learning (TD) is a **bootstrapping** method: it learns directly from transitions of incomplete episodes using previous estimates.

Prediction

For the prediction problem TD updates its estimate of $v^\pi(s_t)$ only using the next step reward r_{t+1} and its current estimate of $v^\pi(s_{t+1})$.

$$V(S_t) \leftarrow V(S_{t+1}) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (1.6)$$

$R_{t+1} + \gamma V(S_{t+1})$ is called TD *target* and the quantity $\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called TD *error*.

Control

For the Control problem TD algorithms can be divided into two categories:

1. On-Policy Learning
2. Off-Policy Learning

In On Policy Learning the algorithm directly improves the policy π used to collect the samples from the environment while Off Policy Learning allows to improve a policy π' different from the one that interacts with the environment.

One major problem in Model-Free reinforcement learning is the *Exploration-Exploitation* tradeoff: Should the agent try out new actions to discover their potential rewards or exploit its current knowledge in order to maximize the expected reward?

Definition 1.3.1. A *soft policy* π is a policy with $\pi(a|s) > 0, \forall s \in \mathcal{S}$ and $\forall a \in \mathcal{A}(s)$.

Definition 1.3.2. An ϵ -*soft policy* π is a policy with $\pi(s, a) \geq \frac{\epsilon}{|\mathcal{A}(s)|}, \forall s \in \mathcal{S}$ and $\forall a \in \mathcal{A}(s)$ and some $\epsilon > 0$.

Definition 1.3.3. An ϵ -*greedy policy* π is a policy with:

$$\pi(a|s) = \begin{cases} \frac{\epsilon}{|\mathcal{A}|} + 1 - \epsilon & \text{if } a = \arg \max_{a' \in \mathcal{A}} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$$

The ϵ -greedy policy is a specific example of ϵ -soft policy that is the closest to a greedy policy. It is the simplest idea for ensuring continual exploration while acting greedily.

Theorem 1.3. For any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to q^π is an

improvement.

$$\begin{aligned}
 q^\pi(s, \pi'(s)) &= \sum_{a \in \mathcal{A}} \pi'(a|s) q^\pi(s, a) \\
 &= \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} q^\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} q^\pi(s, a) \\
 &\geq \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} q^\pi(s, a) + (1 - \epsilon) \sum_{a \in \mathcal{A}} \frac{\pi(a|s) - \frac{\epsilon}{m}}{1 - \epsilon} q^\pi(s, a) \\
 &= \sum_{a \in \mathcal{A}} \pi(a|s) q^\pi(s, a) = v^\pi(s)
 \end{aligned}$$

with $m = |\mathcal{A}|$.

Therefore from the policy improvement theorem 1.2, $v^{\pi'}(s) \geq v^\pi(s)$.

Sarsa: On-policy TD Control

Applying the same TD method (1.6) used to estimate v^π to the action-value function q^π leads to the following update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

.

This rule uses every element of a quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state-action pair to the next. This quintuple gives rise to the name *Sarsa* for the algorithm.

It is straightforward to design an on-policy control algorithm based on the Sarsa prediction method. As in all on-policy methods, we continually estimate q^π for the behavior policy π , and at the same time change π towards greediness with respect to q^π .

Algorithm 1.4 Sarsa

```

1: Initialize  $Q(s, a)$  arbitrarily
2: loop
3:   Initialize  $S$ 
4:   Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
5:   repeat
6:     Take action  $A$ , observe  $R, S'$ 
7:     Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
8:      $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$ 
9:      $S \leftarrow S'; A \leftarrow A'$ 
10:  until  $S$  is terminal
11: end loop

```

Q-Learning: Off-policy TD Control

One of the early breakthroughs in reinforcement learning was the development of an off-policy TD control algorithm known as *Q-Learning* [19].

In this algorithm the learned action-value function Q directly approximates q^* , the optimal action-value function, independent of the policy being followed:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t)]$$

In this case, the learned action-value function Q directly approximates q^* , the optimal value action-value function thanks to the max operation.

Algorithm 1.5 Q-Learning

```

1: Initialize  $Q(s, a)$  arbitrarily
2: loop
3:   Initialize  $S$ 
4:   Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
5:   repeat
6:     Take action  $A$ , observe  $R, S'$ 
7:      $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a \in \mathcal{A}} Q(S', a) - Q(S, A)]$ 
8:      $S \leftarrow S'$ 
9:   until  $S$  is terminal
10: end loop

```

Convergence of Sarsa and Q-Learning

Definition 1.3.4. A learning policy is called *Greedy in the limit of infinite exploration (GLIE)* if it satisfies the following two conditions:

1. All state-action pairs are explored infinitely many times

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

2. The policy converges on a greedy policy

$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbb{1}(a = \arg \max_{a' \in \mathcal{A}} Q_k(s, a'))$$

Theorem 1.4. SARSA converges to the optimal action-value function $Q(s, a) \rightarrow Q^*(s, a)$ under the following conditions:

1. GLIE sequence of policies $\pi_t(s, a)$
2. Robbins-Monro sequence of step-sizes α_t

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

This means that in order for SARSA to converge to q^* we need to decrease ϵ over time (e.g. $\epsilon = \frac{1}{t}$). If ϵ is kept fixed SARSA will converge to the optimal policy among the ϵ -soft policies. Being Q-Learning an off-policy algorithm that is already improving Q towards q^* there is no need for an ϵ decay as long as all state-action pairs are visited infinitely and the learning rate' schedule satisfies the Robbins-Monro conditions.

1.3.2. Reinforcement Learning Taxonomy

This subsection includes a brief overview of the whole Reinforcement Learning taxonomy. 1.3.1 presented two well known TD tabular algorithms but often real world scenarios have to be modeled with complex state and action spaces with more dimensions resulting in intractable problems.

- **Function Approximation**

When the state space is too big for tabular reinforcement learning approximate solution methods are used instead. Typically the value function (or action-value

function) is parameterized with a weight vector $\mathbf{w} \in \mathbb{R}^d$ and $d \ll |\mathcal{S}|$:

$$\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$$

For example $\hat{v}$ might be a linear function in features of the state, with $\mathbf{w}$ the vector of feature weights. More generally, $\hat{v}$ might be the function computed by a multi-layer artificial neural network or even the function computed by a decision tree.

In addition, function approximation allow continuous state spaces to be managed without the need for discretization techniques.

- **Value-Based Algorithms**

All the algorithms that seek to learn the action values and then select actions based on their estimated values are called Value-Based Algorithms. The temporal difference algorithms presented in 1.3.1 are all value-based algorithms.

- **Policy-Based Algorithms**

Differently from Value-Based Algorithms Policy-Based algorithms seek to learn a *parameterized policy* that can select actions without consulting a value function:

$$\pi(a|s, \theta) = \mathbb{P}(A_t = a | S_t = s, \theta_t = \theta)$$

One important consequence of the use of parameterized policies is that they offer practical ways of dealing with large action spaces, even continuous spaces with an infinite number of actions. Instead of computing the learned probability for each of the many actions, we instead learn statistics of the probability distribution. For example, the action set might be the real numbers, with actions chosen from a normal distribution:

Definition 1.3.5. *A gaussian policy suitable for continuous action space is given by*

$$\pi(a|s, \theta) = \frac{1}{\sigma(s, \theta)\sqrt{2\pi}} \exp\left(-\frac{(a - \mu(s, \theta))^2}{2\sigma(s, \theta)^2}\right) \quad (1.7)$$

where $\mu : \mathcal{S} \times \Theta \rightarrow \mathbb{R}$ is the mean function and $\sigma : \mathcal{S} \times \Theta \rightarrow (0, \infty)$ is the standard deviation function. Θ is the parameter space of the policy.

A sample from a gaussian policy can be computed as follows:

$$a = \mu(s, \theta) + \sigma(s, \theta)\eta \quad (1.8)$$

drawing η from a standard distribution $\mathcal{N}(0, 1)$.

- **Actor-Critic Algorithms**

A combination of value-based methods and policy-based methods gives rise to Actor-Critic algorithms. Actor critic algorithms learn a parameterized policy (Actor) and a value function (Critic). The actor is updated in the direction that maximizes the expected reward using gradients derived from the critic's feedback. The critic estimates the actions taken by the actor.

1.3.3. Deep Reinforcement Learning

In recent years Deep Learning techniques applied to Reinforcement Learning techniques have shown to yield very good results. The first notable example came from Mnih et al. [12] modifying Q-Learning to work with Deep Neural Networks. At that time, their approach achieved state of the art on multiple Atari Games with no prior knowledge and same hyperparameters for each game. This algorithm, called *Deep Q-Networks* (DQN) uses an *experience replay buffer* to store played transitions for sampling minibatches. These minibatches are used to train the Q-Network using Stochastic Gradient Descent with backpropagation on the neural network.

Unfortunately nonlinear function approximators like neural networks come with a theoretical cost: convergence to an optimum is only guaranteed locally. Despite this limitation, deep reinforcement learning approaches remain the most widely used methods in the field due to their empirical success and ability to handle complex, high-dimensional state spaces.

Below, a state of the art algorithm called *Soft Actor-Critic* (SAC) is presented.

Soft Actor-Critic

SAC [4, 5] is an off policy actor-critic deep RL algorithm based on the maximum entropy framework in which the standard reinforcement learning objective is augmented with an entropy term:

$$\pi^* = \arg \max_{\pi} \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

where α is the temperature parameter that determines the relative importance of the entropy term versus the reward, and thus controls the stochasticity of the optimal policy. This objective has shown to improve exploration and improve learning speed over state-of-art methods that optimize the conventional RL objective function.

Definition 1.3.6. *The soft state-value and action value functions $v_{\text{soft}} : \mathcal{S} \rightarrow \mathbb{R}$ and $q_{\text{soft}} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ are defined by:*

$$\begin{aligned} v_{\text{soft}}^\pi(s_t) &= \mathbb{E}_{a_t \sim \pi} [q^\pi(s_t, a_t) - \alpha \log \pi(a_t | s_t)] \\ q_{\text{soft}}^\pi(s_t, a_t) &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [v_{\text{soft}}^\pi(s_{t+1})] \end{aligned}$$

Soft Actor-Critic is based on the soft policy iteration algorithm, an adaptation of policy iteration algorithm that includes the entropy term in the value function and guarantees convergence to an optimal policy.

In the soft policy iteration, the policy is updated towards the exponential of the soft Q-function. This particular choice of update can be guaranteed to result in an improved policy in terms of its soft values:

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{KL} \left(\pi'(\cdot | s_t) \left\| \frac{\exp(\frac{1}{\alpha} Q^{\text{old}}(s_t, \cdot))}{Z^{\pi_{\text{old}}}(s_t)} \right\| \right), \quad (1.9)$$

where the partition function $Z_{\text{old}}^\pi(s_t)$ normalizes the distribution, and while it is intractable in general, it does not contribute to the gradient with respect to the new policy and thus can be ignored.

Soft Actor-Critic uses function approximators for both the soft Q-function and the policy, and instead of running evaluation and improvement to convergence like soft policy iteration, alternates between optimizing both networks with stochastic gradient descent. The soft Q-function can be modeled as expressive neural networks, and the policy as a Gaussian distribution (1.7) with mean and covariance given by neural networks

The soft Q-function parameters can be trained to minimize the soft Bellman residual through stochastic gradients:

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - \left(r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V_{\bar{\theta}}(s_{t+1})] \right) \right)^2 \right]$$

where $\mathcal{D}$ is the replay buffer storing past experience. In this way the reinforcement learning objective is approximated as a supervised learning objective over a continually

changing buffer of past experience.

$$\hat{\nabla}_{\theta} J_Q(\theta) = \nabla_{\theta} Q_{\theta}(a_t, s_t) (Q_{\theta}(s_t, a_t) - (r(s_t, a_t) + \gamma (Q_{\bar{\theta}}(s_{t+1}, a_{t+1}) - \alpha \log(\pi(a_{t+1}|s_{t+1}))))))$$

The update makes use of a target Q-Function with parameters $\bar{\theta}$ that are obtained as an exponential moving average of the soft Q-function weights, which has shown to stabilize training [12]. Finally, the policy parameters can be learned by directly minimizing the expected KL-Divergence in (1.9):

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}} [\mathbb{E}_{a_t \sim \pi_{\phi}} [\alpha \log \pi_{\phi}(a_t|s_t) - Q_{\theta}(s_t, a_t)]] \quad (1.10)$$

As the Q-function is differentiable it is convenient to apply the reparameterization trick to the policy in order to minimize J_{π} :

$$a_t = f_{\phi}(\epsilon_t; s_t)$$

where ϵ_t is an input noise, sampled from some fixed distribution, such as a spherical Gaussian.

The objective function in (1.10) can be rewritten as

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}} [\alpha \log \pi_{\phi}(f_{\phi}(\epsilon_t; s_t)|s_t) - Q_{\theta}(s_t, f_{\phi}(\epsilon_t; s_t))] \quad (1.11)$$

where π_{ϕ} is implicitly defined in terms of f_{ϕ} . The gradient of equation (1.11) is then approximated with

$$\hat{\nabla}_{\phi} J_{\pi}(\phi) = \nabla_{\phi} \alpha \log \pi_{\phi}(a_t|s_t) + (\nabla_{a_t} \alpha \log \pi_{\phi}(a_t|s_t) - \nabla_{a_t} Q_{\theta}(s_t, a_t)) \nabla_{\phi} f_{\phi}(\epsilon_t; s_t)$$

where a_t is evaluated at $f_{\phi}(\epsilon_t; s_t)$.

SAC also employs an automatic entropy adjustment algorithm that finds the optimal entropy coefficient during training. Also, it makes use of two soft Q functions to mitigate positive bias in the policy improvement step that is known to degrade performance of value based methods [3, 6]. In particular, we parameterize two soft Q-functions, with parameters θ_i , and train them independently to optimize $J_Q(\theta_i)$. For further details refer to [5].

Algorithm 1.6 Soft Actor-Critic (SAC) Algorithm

```

1: Initialize policy parameters  $\phi$ , Q-function parameters  $\theta_1, \theta_2$ 
2: Initialize replay buffer  $\mathcal{D}$ 
3: Set target Q-function weights:  $\bar{\theta}_i \leftarrow \theta_i$  for  $i \in \{1, 2\}$ 
4: for each iteration do
5:   for each environment step do
6:     Sample action  $a_t \sim \pi_\theta(a_t|s_t)$ 
7:     Execute action  $a_t$  in the environment
8:     Observe next state  $s_{t+1}$ , reward  $r_t$ 
9:     Store  $(s_t, a_t, r_t, s_{t+1})$  in replay buffer  $\mathcal{D}$ 
10:  end for
11:  for each gradient step do
12:    Sample a batch of transitions  $(s, a, r, s')$  from  $\mathcal{D}$ 
13:     $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$  for  $i \in \{1, 2\}$ 
14:     $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ 
15:     $\bar{\theta}_i \leftarrow \tau \theta_i + (1 - \tau) \bar{\theta}_i$  for  $i \in \{1, 2\}$ 
16:  end for
17: end for

```

2 | Related Works

In this chapter, we review an optimization-based approach to air hockey trajectory planning and examine the main challenges in applying reinforcement learning within robotics. We also describe a state-of-the-art algorithm for safe exploration.

2.1. Controller for the Air Hockey task

In robotics, the desired behavior of a robot is often defined through planned trajectories with specific structures, such as cubic splines. These trajectories are then optimized based on various criteria, including path smoothness, proximity to obstacles, travel time, path length, and adherence to the robot's constraints.

An example of these techniques is given by [10], where the authors implemented an algorithm for the Air Hockey hit task described in 3.1.2 using a general purpose robot with 7 DoFs. The end-effector's trajectory for hitting the puck is planned avoiding collisions and then optimized with multiple optimization problems in order to achieve high performance while adhering to the constraints :

Weighted Null Space Optimization: To achieve high speed trajectories, the null space velocity is optimized at every point of the trajectory. Considering only the position of the end-effector in task space, the concerned Jacobian is $J_p \in \mathbb{R}^{3 \times n}$, where n is the number of joints. Let $b = J_p^\dagger \left(\frac{\bar{x} - x}{\Delta T} + \bar{v} \right)$ be the joint velocities for the trajectory tracking with $J_p^\dagger = J^\top (J J^\top)^{-1}$ being the general pseudoinverse of J_p , x , $\bar{x}$, $\bar{v}$ are respectively actual, desired cartesian position and desired cartesian velocity. The objective is to maximize the weighted joint velocity as

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \|b + E_N \alpha\|_W^2 \\ \text{s.t.} \quad & \dot{q}^{\min} \leq b + E_N \alpha \leq \dot{q}^{\max} \end{aligned}$$

where $E_N \in \mathbb{R}^{n \times (n-3)}$ is the *null space matrix*. Each column of the *null space matrix* is one basis vector of $\text{Null}(J_p)$ and $\alpha \in \mathbb{R}^{n-3}$ are entries on each basis vector. It should be

pointed out that, when W is set to be an identity matrix, the optimal solution is $\alpha = 0$ when the constraints are not violated. In practice higher weights are applied on joints that afford more loads and have lower velocity limits. The next step and velocity are obtained by Euler integration

$$\begin{aligned}\dot{q}_{next} &= b + E_N \alpha^* \\ q_{next} &= q_{current} + \dot{q}_{next} \Delta T\end{aligned}$$

Hitting Configuration Optimization: To find the best configuration for the hitting movement, the manipulability along a specific hitting direction v is considered, as presented in [18]. The following quantity at the hitting point p is maximized

$$\begin{aligned}\max_q \quad & \| (v^\top J_p(q)) \|_2, \\ \text{s.t.} \quad & FK_p(q) = p\end{aligned}$$

where $FK_p(q)$ is the forward kinematic w.r.t the end-effector position.

Computing the maximum hitting velocity: To compute the maximum hitting velocity at the selected hitting configuration, two strategies can be used:

1. *Least Square solution:* The hitting speed velocity is parameterized as a scalar η and a hitting direction v , such that $v_{\max} = \eta v$. The least square solution is $\dot{q} = J_p^\dagger(q^*) v_{\max}$. Thus, the maximum possible joint velocity can be determined by the minimum ratio of the absolute value of the i -th joint velocity $\dot{q}_i$ and its maximum $\dot{q}_i^{\max}$

$$\eta = \min_i \left(\frac{\dot{q}_i^{\max}}{|\dot{q}_i|} \right), \quad i \in \{1, \dots, n\}.$$

2. *Linear Programming:* Instead of using the least squared solution, a linear program on the null space can be constructed as

$$\begin{aligned}\max_{\alpha} \quad & v^\top J_p(q^*) (E_{v^\perp}(q^*) \alpha), \\ \text{s.t.} \quad & \dot{q}^{\min} \leq E_{v^\perp}(q^*) \alpha \leq \dot{q}^{\max},\end{aligned}$$

where $v^\perp \in \mathbb{R}^3$ is the orthogonal complement space of $v \in \mathbb{R}^3$, which in this case is a plane orthogonal to the hitting direction, and $E_{v^\perp}(q^*) = \text{Null} [(v^\perp)^\top J_p(q^*)]$ is the basis vectors spanning the null space of $(v^\perp)^\top J_p(q^*)$.

2.2. Reinforcement Learning in Robotics

Reinforcement Learning enables robots to learn complex behaviors through interaction with the environment and maximizing the cumulative rewards. This can be particularly advantageous for tasks that are too intricate or dynamic to be effectively addressed by conventional methods. On the other hand traditional robotics techniques, such as classical control theory and model-based approaches, are typically more predictable and easier to implement. While these techniques may lack the flexibility and adaptability of reinforcement learning, they offer a more straightforward and less resource-intensive path to deploying functional robotic systems. Reinforcement Learning in Robotics has however many challenges to face [7], and these are described in the following section.

Curse of Dimensionality

Robotics systems often have to deal with high dimensional state and action spaces due to the many degrees of freedom (DoFs) of modern anthropomorphic robots. For example, in the ball-paddling task shown in Figure 2.1, where a robotic arm needs to maintain the ball bouncing on a racket, a proper representation of a robot's state would consist of its joint angles and velocities for each of its seven DoFs as well as the cartesian position and velocity of the ball. The robot's actions would be the generated motor commands, which often are torques or accelerations.

This curse of dimensionality presents significant challenges in terms of computational complexity and the effectiveness of learning algorithms.

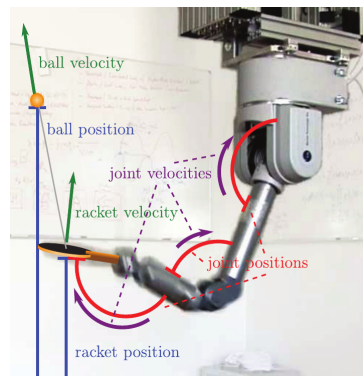


Figure 2.1: Ball Paddling Robot

This problem is solved with function approximators like neural networks using Deep Reinforcement Learning algorithms.

Curse of real-world samples

Robots interact with the physical world, hence robot reinforcement learning suffers from most of the resulting real-world problems:

- **Wear and tear:** Robot hardware is usually expensive, suffers from wear and tear, and requires careful maintenance. This is why safe exploration becomes a key issue in the learning process.
- **Reproducibility:** Several more aspects of the real-world make robotics a challenging domain. Frequently, the environment settings during an earlier learning period cannot be reproduced. This problem makes comparing algorithms particularly hard. Furthermore, the approaches often have to deal with uncertainty due to inherent measurement noise and the inability to observe all states directly with sensors.
- **Human supervision:** Most real robot learning tasks require some form of human supervision, e.g. putting the pole back on the robot's end-effector during pole balancing after a failure. Even when an automatic reset exists, the learning speed becomes essential as a task on a real robot cannot be sped up. In some tasks such as a slowly rolling robot, the dynamics can be ignored; in others such as a flying robot, they cannot. In particular in the latter case, often the whole episode needs to be completed as it is not possible to start from arbitrary states.
- **Real-Time constraints:** Since the robot is a physical system, there are strict constraints on the interaction between the learning algorithm and the robot setup. For dynamic tasks, the movement cannot be paused and actions must be selected within a time-budget without the opportunity to pause to think, learn or plan between actions.
- **Discretization and delay:** As reinforcement learning algorithms are inherently implemented on a digital computer, the discretization of time is unavoidable despite the inherently continuous time configuration of physical systems. Time-discretization of the actuation can generate undesirable artifacts. As most robots are controlled at fixed sampling frequencies (in the range between 500 Hz and 3 kHz) determined by the manufacturer of the robot, the upper bound on the rate of temporal discretization is usually pre-determined. The lower bound depends on the horizon of the problem, the achievable speed of changes in the state, as well as delays in sensing and actuation. All physical systems exhibit such delays in sensing and actuation. The state of the setup (represented by the filtered sensor signals) may frequently lag behind the real state due to processing and communication delays. More crit-

ically, there are also communication delays in actuation as well as delays due to the fact that neither motors, gear boxes nor the body's movement can change instantly. Owing to these delays, actions may not have instantaneous effects but are observable only several time steps later. In contrast, in most general reinforcement learning algorithms, the actions are assumed to take effect instantaneously as such delays would violate the usual Markov assumption. This effect can be addressed by putting some number of recent actions into the state. However, this significantly increases the dimensionality of the problem. The problems related to time budgets and delays can also be avoided by increasing the duration of the time steps. One downside of this approach is that the robot cannot be controlled as precisely.

For such reasons, real-world samples are expensive in terms of time, labor and, potentially, finances. In robotic reinforcement learning, it is often considered to be more important to limit the real-world interaction time instead of limiting memory consumption or computational complexity. Thus, sample efficient algorithms that are able to learn from a small number of trials are essential.

Curse of under-modeling and model uncertainty

One way to offset the cost of real-world interaction is to use accurate models as simulators. In an ideal setting, this approach would make it possible to learn the optimal policy within the emulated environment and subsequently transfer it to the real robot. Unfortunately, creating a sufficiently accurate model of the robot and its environment is challenging and often requires very many data samples. As small model errors due to this under-modeling accumulate, the simulated robot can quickly diverge from the real-world system.

For tasks where the system is self-stabilizing (that is, where the robot does not require active control to remain in a safe state or return to it), transferring policies often works well. Nevertheless, tasks can often be learned better in the real world than in emulated environments due to complex mechanical interactions that have proven difficult to model accurately. In contrast, in unstable tasks small variations have drastic consequences.

Curse of goal specification

In reinforcement learning, the desired behavior is implicitly specified by the reward function. The goal of reinforcement learning algorithms then is to maximize the accumulated long-term reward. In practice, it can be surprisingly difficult to define a good reward function in robot reinforcement learning. In many domains, it seems natural to provide rewards only upon task achievement, for example, when a table tennis robot wins a match.

This view results in an apparently simple, binary reward specification. However, a robot may receive such a reward so rarely that it is unlikely to ever succeed in the lifetime of a real-world system. Instead of relying on simpler binary rewards, we frequently need to include intermediate rewards in the scalar reward function to guide the learning process to a reasonable solution, a process known as *reward shaping* [8]. This approach can add additional guidance to the agent, helping it learn the desired behavior faster by indicating that certain actions are beneficial. However *reward shaping* requires expert knowledge to ensure the rewards accurately reflect progress towards the goal.

2.3. ATACOM

One important factor that cannot be neglected in real-world applications is the necessity of satisfying constraints. Many practical considerations can be formulated in the form of constraints, such as safety and mechanical viability. For example, in the robot manipulation task, the robot should not take actions that damage the environment and cannot take actions that exceed its feasible range of motion.

In order to achieve safe exploration during learning process in continuous control problems different algorithms have been developed. This section describes the one that was employed in this thesis, ATACOM [11]

ATACOM transforms a constrained RL problem into a typical unconstrained RL problem. Hence the problem can be addressed by any model-free RL algorithm while maintaining the constraints below the tolerance. Furthermore, ATACOM can handle both equality and inequality constraints.

The state variable $s \in \mathcal{S}$ is decomposed in the directly controllable state $q \in \mathcal{Q}$ and uncontrollable state, i.e., $s = [\mathbf{q} \ \mathbf{x}]^\top$. We assume that the constraints $\mathbf{c}(\mathbf{q}) \leq \mathbf{0}$ are known and depend purely on the controllable state.

The idea behind ATACOM is to construct the constraint manifold $\mathcal{M}_c : \mathbf{c}(\mathbf{q}) = \mathbf{0}$, determine the bases $\mathbf{N}_c$ of the tangent space $\mathcal{T}_c$ and sample state velocity in the tangent space (Figure 2.2).

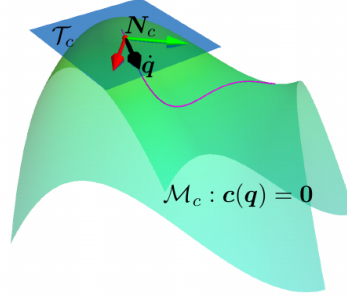


Figure 2.2: Acting on the Tangent Space of the Constraint Manifold. The constraint set $c(\mathbf{q}) = \mathbf{0}$ is a differentiable manifold $\mathcal{M}_c$ embedded in the original state space. A set of basis N_c is used to represent the span of tangent space $\mathcal{T}_c$. The tangent space velocity/acceleration can be determined by a coordinate based on the basis, and the control action is determined based on the tangent space velocity/acceleration, the resulting trajectory is maintained on the constraint manifold.

The advantages of ATACOM can be summarized as follows:

- It can deal both with **equality and inequality constraints**. All of the constraints at each time step are maintained below the tolerance during the whole process.
- It does not require an initial feasible policy, the agent can **learn from scratch**.
- It requires **no manual safe backup policy** to move the system back into the safe region.
- It can be applied to any model-free RL algorithm, using both **deterministic and stochastic policies**
- It can focus the exploration on the **lower-dimensional manifold** instead of exploring in the original action space for equality constrained problem.
- It has **better learning performance** as the inequality constraints restrict to a smaller feasible state-action space.

As a downside this method requires

- differentiable constraint functions.
- a sufficiently accurate invertible dynamics model of the robot or a well-performed tracking controller.

3 | Robot Air Hockey Challenge

This chapter describes the Robot Air Hockey competition that took place from 20 February 2023 to 1 November 2023. It covers the game rules, the motivation behind the event, the various stages of the competition and the outcomes achieved. The following information is derived from the challenge's website [1]

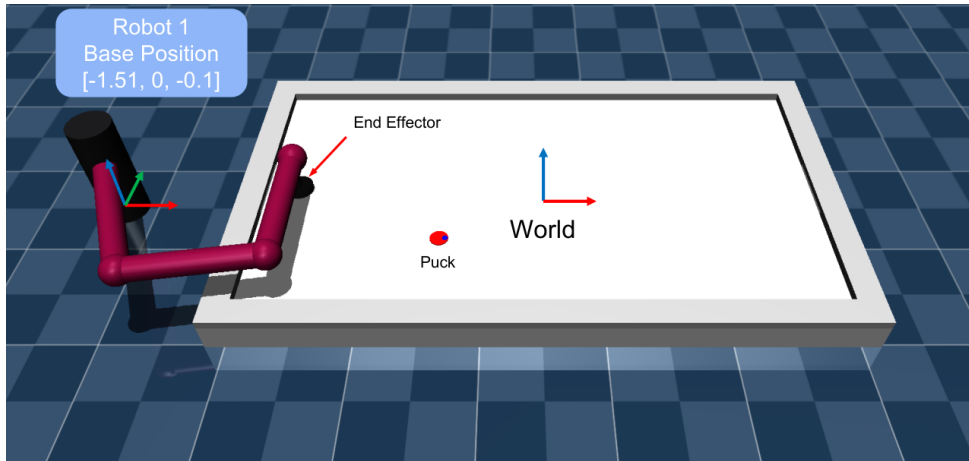


Figure 3.1: Planar robot base frame.

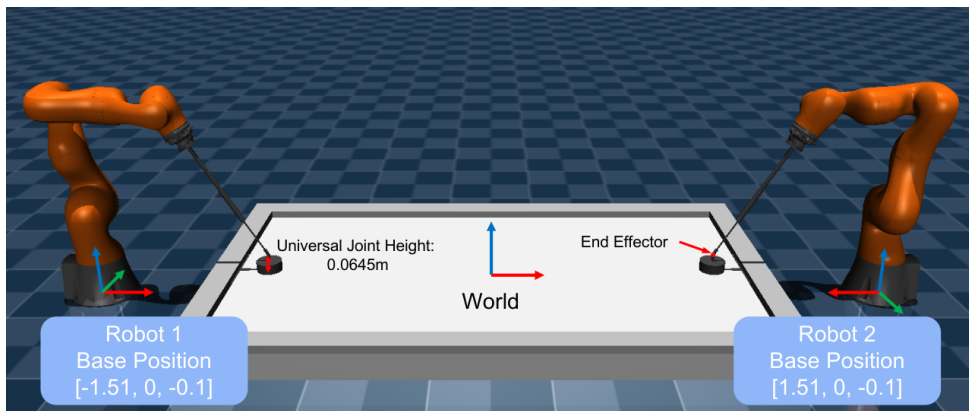


Figure 3.2: KUKA robot base frame.

3.0.1. Air Hockey Game

Air Hockey is a zero sum game in which two opponents compete interacting with a puck on a table and trying to score a goal sending the puck into the opponent's goal while at the same time preventing the opponent from scoring. The table is rectangular and smooth, with low-friction achieved by air blown from under the surface, designed to mimic the surface of ice. This game is dynamic and the puck reaches high velocities that require the players to have quick reflexes and strategic positioning.

3.0.2. Motivation

This competition was organized to *close the sim-to-real gap*, i.e., the differences and challenges that arise when transferring a robotic system trained in a simulated environment to the real world. In fact, when transferring the robotic system to the real world we have to deal with

- **Disturbances:** Real-world environments are subject to various unpredictable disturbances, such as vibrations, temperature variations, and other environmental factors that can affect the performance of the robotic system.
- **Observation noise:** Sensors used in real-world applications often introduce noise into the observed data, which can lead to inaccuracies in the robot's perception of its environment and subsequent decision-making processes.
- **Safety:** Ensuring the safety of both the robotic system and its surroundings, including humans and other objects, is paramount. The robot must operate within safe parameters to avoid causing harm or damage.
- **Model mismatch:** The physical properties and dynamics of real-world systems often differ from their simulated counterparts. This mismatch can lead to discrepancies in performance and behavior when the robot is deployed outside the controlled environment of a simulation.
- **Delays:** Communication and processing delays can occur in real-world systems, affecting the timing and execution of the robot's actions. These delays need to be accounted for to ensure smooth and accurate operation.
- **Actuator limitations:** The actuators used to control the robot have physical limitations in terms of speed, precision, and strength. These constraints must be considered when designing control strategies and algorithms.
- **Limited real-world interaction:** In the real world, opportunities for interaction

and experimentation are often limited due to cost, time, and safety concerns. This restriction makes it challenging to gather sufficient data for training and refining the robotic system.

The Robot Air Hockey Challenge provides a platform for researchers in the field of robot learning to interact with each other on a realistic robotic task. In this competition team had to design and build their air hockey agents, competing against each other in different subtasks (in simulation) and finally in an entire game (both in simulation and in the real world).

3.1. Challenge organization

The challenge consists in three simulated stages: Warm Up, Qualifying and Tournament. Each stage consists of different tasks required for robot air hockey. Apart from executing the given tasks the robots need to be robust and capable of adapting to changes. Except from the Warm Up stage the submitted agents are evaluated in modified simulation environments to mimic the sim-to-real gap.

The first three teams are able to deploy their approach on the real robot and compete in a real world scenario.

3.1.1. Warm-Up Stage

In the Warm-Up stage the participants were given an ideal environment with a 3-degree-of-freedom robot. This stage aimed to familiarize with the given tasks, the environment and the application programming interface.

The tasks given at this point were two:

- **Hit:** The puck is randomly initialized on the left side of the table. The initial velocity is zero. The objective is to hit the puck to score a goal as fast as possible.
- **Defend:** The puck is randomly initialized on the right side of the table with a random velocity heading left. The objective is to stop the puck on the right side of the table and prevent it from getting scored.

3.1.2. Qualifying Stage

In this stage the participants were given an environment exposing an interface to control a general-purpose robot, the KUKA-iiwa14 LBR. At this point the submitted agents to

the challenge’s cloud evaluator were simulated with a modified version of the environment handed to the participant teams in order to simulate different types of real world problems. This modifications included disturbances, observation noise, loss of tracking, model mismatch, and imperfect tracking controller.

Each team could evaluate its solution once per task per day. Each evaluation was conducted with 1000 episodes, equivalently 2.8 hours of real-world experiments. Based on the evaluation metric, the agents were categorized into three levels:

- Deployable
- Improvable
- Nondeployable.

The given tasks at this stage were three:

- **Hit:** The opponent moves in a predictable pattern. The puck is randomly initialized with a small velocity. The objective is to score the goal as many times as possible. The episode ends when the puck is scored or it bounces back. The episodes terminates with *success* if the puck is scored.
- **Defend:** The puck is randomly initialized on the right side of the table with a random velocity heading left. The objective is to stop the puck on the right side of the table and prevent it from getting scored. The episode terminates if the puck is returned to the opponent’s side or scored or the puck speed drops below the threshold. The episode terminates with *success* if the puck is in the range where hits can be made and the longitudinal speed is below the threshold.
- **Prepare:** The puck is initialized close to the table’s boundary and is unsuitable for hitting. The task is to control the puck to move it into a good hit position. The puck is not allowed to cross the middle line. The episodes terminates if the puck crosses the middle line that connects the middle points of two goals, or the puck is on the opponent’s side of the table. The episodes terminates with success if the puck is in the range where its can be made and the longitudinal speed is below the threshold.

3.1.3. Tournament Stage

Only the participant teams who submitted agents that were categorized as Deployable or Improvable were qualified to participate in the tournament stage. The maximum number of teams is 16, which were determined based on the ranking of *success rate* in the qualifying

stage. In this stage each team had to develop an agent able to play the whole game. A hard coded baseline based on the implementation in [10] agent was provided to test and validate the agents before the actual competition started.

The tournament stage was divided into two rounds. After the first rounds teams had two weeks to adjust and improve their agent. Teams were awarded three points if they won a match, otherwise one point if they drew and zero if they lost. The final ranking of the tournament was determined by the total score of the two rounds.

Game Rules

- A game is 15 minutes in length, i.e. 45000 steps.
- The first player to accumulate seven (7) points wins the game. If no player accumulates seven points within the designated competition time, the player with the most points wins.
- To score a point, the puck must fully enter the goal. Rebounds or pucks that get stuck halfway in do not count as a point.
- When a player makes a goal, the other player serves the puck next.
- A player may only strike the puck when it is on their side of the centerline.
- Mallets may not cross the center line when striking the puck.
- When the puck stops in the middle of the table (20cm width), the puck is reset randomly to one player's side
- "Topping" the puck is not allowed. This means players cannot lift their mallet and place it over the puck to hold it in place.
- Touching the puck with other parts of the robot is not allowed.
- Once the puck is on a certain player's side of the center line, the player has 15 seconds, i.e. 750 steps, to hit the puck back across the center line. Otherwise a foul is committed and the opponent receives possession of the puck.
- If a player commits three fouls a point is given to the opponent.

3.2. Framework

The Air Hockey Challenge is built upon MushroomRL [2], a Reinforcement Learning Library. The general framework of challenge consists of two key components, i.e., *Agent*

and *AirHockeyChallengeWrapper*. A schema of the framework can be seen in Figure 3.3. Participants had to implement the *Agent* component.

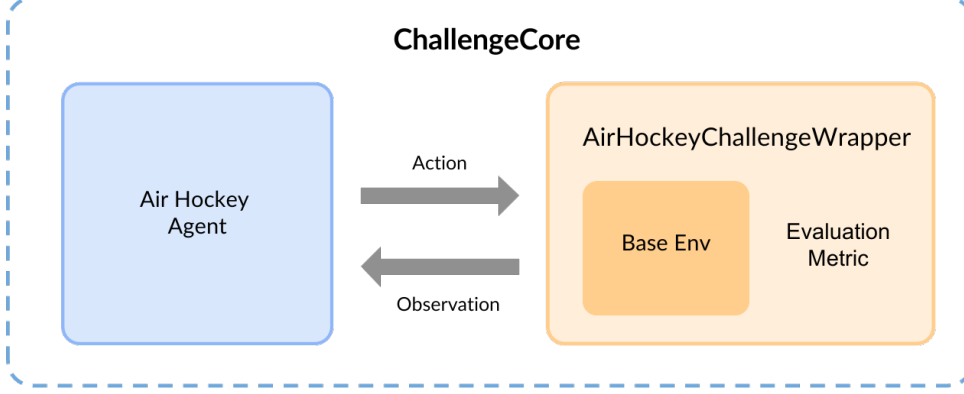


Figure 3.3: Challenge framework.

3.2.1. Environments

The *AirHockeyChallengeWrapper* component is a wrapper for the *Base env* and processes the necessary information for the challenge evaluation.

The simulated environment tries to mimic the real-world setup. The robot is controlled by a *Tracking controller*, a Feedforward-PD controller, which sends the torque command τ_{cmd} to the robot.

$$\tau_{cmd} = M(q)\ddot{q}_d + c(q, \dot{q}) + g(q) + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q})$$

where $M(q)\ddot{q}$ is the inertia matrix, $c(q, \dot{q})$ represents the Coriolis and centrifugal forces and $g(q)$ represents the gravitational forces acting on the robot. Finally, K_p is the proportional gain matrix and K_d is the derivative gain matrix.

The *Trajectory interpolator*, by default a cubic polynomial, is used to interpolate the trajectory points between two consecutive commands. A *Joint safety limiter* is used to avoid executing commands that exceed the position or velocity limits.

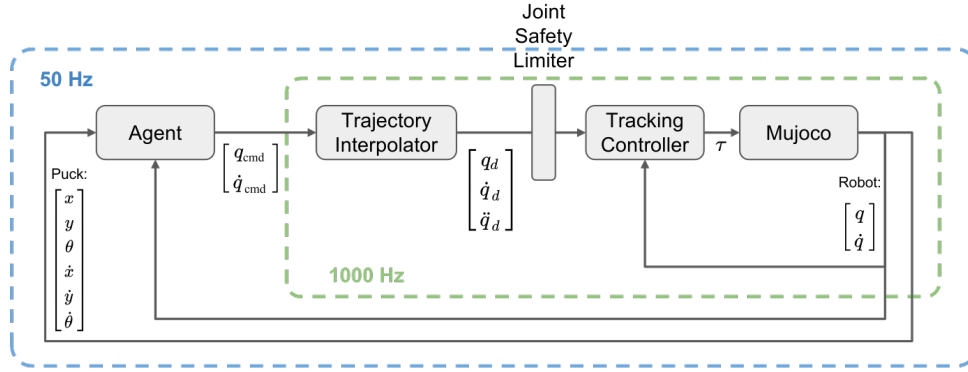


Figure 3.4: Control loop.

The framework provides a flexible interface for commanding the robot in which the trajectory interpolation order can be specified:

- Cubic interpolation. The action command contains the desired [position, velocity]. A cubic polynomial is used to interpolate the intermediate steps.
- Linear interpolation. The action command contains the desired [position]. A linear polynomial is used to interpolate the intermediate steps.
- Quadratic interpolation. The action command contains the desired [position]. A quadratic function uses the previous position, velocity and the desired position to interpolate the intermediate steps.
- Quartic interpolation. The action command contains the desired [position, velocity]. A quartic function uses the previous position, velocity and the desired position, velocity to interpolate the intermediate steps.
- Quintic interpolation. The action command contains the desired [position, velocity, acceleration]. A quintic function is computed by the previous position, velocity, acceleration and the desired position, velocity and acceleration to interpolate the intermediate steps.
- Linear interpolation in position and velocity. The action command contains the desired [position, velocity]. The position and velocity will both be linearly interpolated. The acceleration is computed based on the derivative of the velocity. This interpolation is not proper, but it is useful to avoid oscillatory in the interpolation.
- None. The action can be a complete trajectory between each action step. At each step, the trajectory command should include desired [position, velocity, acceleration]

The air hockey table and its dimensions are illustrated in Figure 3.5

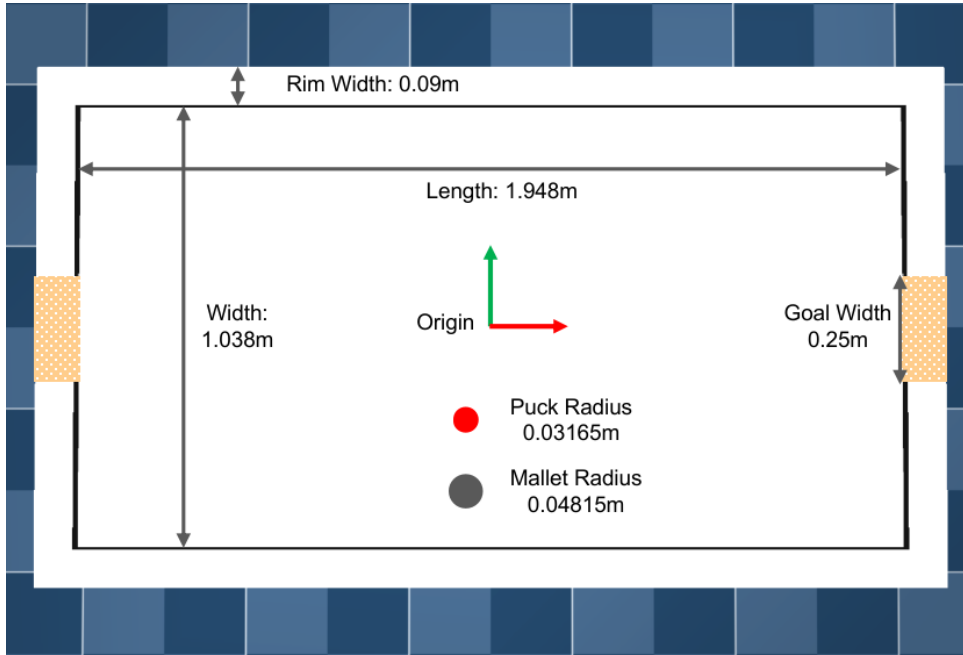


Figure 3.5: The air hockey table.

3.2.2. Robot models

Planar robot - 3 degrees of freedom

During the Warm up stage the participants were provided a 3 DoF planar robot to understand how to use the framework. The robot is set such that the end-effector remains at the same height of the table surface. In simulation, the collision between the robot and the table are discarded. The base of the planar robot is located at $[-1.51, 0.0, -0.1]$, the orientation is aligned with the world's frame, as illustrated in Figure 3.1

Below the specification of the planar robot:

Position upper limit (rad)	2.967060	1.8	2.094395
Position lower limit (rad)	-2.967060	-1.8	-2.094395
Initial position (rad)	-1.156	1.300	1.443
Initial velocity (rad/s)	0	0	0

Table 3.1: 3 DoF planar robot specifications.

KUKA iiwa14 LBR robot

The KUKA iiwa14 LBR robot is a general purpose manipulator with 7 degrees of freedom. A universal joint on the end-effector is added to increase the robot's flexibility. The universal joint is a passive joint that adapts the joint position based on contacts. In simulation, the collision between the robot and the table are discarded, requiring the agent to keep the mallet at an appropriate height.

We define the End-Effector as the tip of the extension rod before the universal joint. The end-effector's position can be fully determined by the robot's joint position and forward kinematics. The base position of the robot is depicted in Figure 3.2.

Below the specification of the planar robot:

Position upper limit (rad)	2.967	2.09	2.967	2.094	2.967	2.094	3.054
Position lower limit (rad)	-2.967	-2.09	-2.967	-2.094	-2.967	-2.094	-3.054
Velocity limit (rad/s)	1.483	1.483	1.745	1.308	2.268	2.356	2.356
Initial position (rad)	0	-0.1960	0	-1.8436	0	0.9704	0
Initial velocity (rad/s)	0	0	0	0	0	0	0

Table 3.2: 7 DoF robot specifications.

3.2.3. Constraints

To avoid losing points the robot must respect a set of constraints. These constraints are described here:

- *JointPositionConstraint*: $q_l < q_{cmd} < q_u$
- *JointVelocityConstraint*: $\dot{q}_l < \dot{q}_{cmd} < \dot{q}_u$
- *EndEffectorConstraint*:

$$l_x < x_{ee}$$

$$l_y < y_{ee} < u_y$$

$$\text{table height} - \text{tolerance} < z_{ee} < \text{table height} + \text{tolerance}$$

The End-Effector must not exceed the table and stay at

- *LinkConstraint* (7 DoF Robot only):

$$z_{elbow} > 0.25$$

$$z_{wrist} > 0.25$$

- *ComputationTime*: The computation time at each step should be smaller than 0.02s.

3.2.4. Evaluation Metrics

- **Success Rate**: A success criterion is defined for each task. We will check if the task succeed when the episode terminates. Each episode can terminate because of two reasons: maximum number of steps reached; no further interaction can be in the episodes.
 - Hit: The puck enters the scoring zone with a speed above the threshold.
 - Defend: The final velocity of the puck (at the end of the game) is below the threshold and does not bounce to the other side of the court.
 - Prepare: The final position of the puck is within a predefined area and at a speed less than the threshold.
- **Deployability**: Each deployability metric is assigned one or multiple penalty points based on the level of risk. The deployability score is counted when the constraints of the evaluation metric are violated. Each metric is computed at most once per episode (maximum 500 steps per episode).
 - Violations of the End-Effector’s Position Constraints (3 penalty points): The desired x-y-position of the end-effector should remain within the boundaries of the table. The z-position of the end-effector should remain within a range.
 - Violations of the Joint Position Limit Constraints (2 penalty points): The desired position command should not exceed the position limits.
 - Violations of the Joint Velocity Limit Constraints (1 penalty point): The desired velocity command should not exceed the velocity limits.
 - Computation Time (0.5-2 penalty points): The computation time at each step should be shorter than 0.02s.

Each evaluation consisted of 1000 episodes. The success rates were used to rank the leaderboards. The deployability score is the sum of the penalty score from all episodes. The rankings were divided into three categories based on the score of deployability.

1. **Deployable:** $0 \leq \text{deployability score} \leq 500$
2. **Improvable:** $500 < \text{deployability score} \leq 1500$
3. **Nondeployable:** $1500 < \text{deployability score}$

3.2.5. Simulator

The simulator application of the Robot Air Hockey Challenge is summarized in the following:

- Simulator: MuJoCo [17]
- Simulation frequency: 1000 Hz
- Control Frequency: 50 Hz
- Observation:
 - **Robot:**
 - * Joint positions [radians]
 - * Joint velocities [radians/s] computed by finite difference
 - **Puck** (relative to the robot base frame):
 - * X-Y position [m]
 - * Yaw angle [radians]
 - * X-Y velocity [m/s]
 - * Yaw velocity [radians/s] computed by finite difference
 - **Opponent** (if applicable)
 - * End-effector-s X-Y position [m]
- Control action:
 - We try to set the simulation as close as possible to the real-world setup. The robot is controlled by torque at 1000 Hz. We use a feed-forward + PD controller to determine the joint torque that tracks the desired joint position and velocity.
 - A 20-step interpolation is required in the agent's adjacent control commands (the agent control frequency is 50 Hz). We provide different types of polynomial interpolation upon requirements. Details about the action interface can be found here.

4 | Methodology

This chapter presents the methodologies employed for the Robot Air Hockey challenge. It starts with a description of how the problem was modeled as an MDP for all the different environments and tasks, and follows with how each of them was addressed.

4.1. Air hockey as an MDP

This section describes how the Air hockey environment was modeled as an MDP by defining the state-action space and the employed reward functions.

4.1.1. State space

The observations provided by the original environment were slightly modified. Indeed the angular position θ and velocity $\dot{\theta}$ of the puck was discarded as well as the z-axis element of the puck's position p_{puck} and velocity $\dot{p}_{puck}$. The end-effector's position p_{ee} and velocity $\dot{p}_{ee}$ were computed using the forward kynematics of the robot:

$$\begin{aligned} p_{ee} &= FK(q) \\ \dot{p}_{ee} &= J(q)\dot{q} \end{aligned}$$

where $q = \{q_i | i \in 1, \dots, 7, q_i^{\min} \leq q_i \leq q_i^{\max}\}$ fully represents the joint angles of the robot, FK is the forward kynematics function and J is the geometric Jacobian matrix of the forward kynematics function. Also the z-axis component of the position and velocity of the end-effector were discarded. The resulting state space is given by

$$s = \begin{bmatrix} p_{puck} & \dot{p}_{puck} & q & \dot{q} & p_{ee} & \dot{p}_{ee} \end{bmatrix}$$

with $s \in \mathbb{R}^{22}$ for the 7 DoF robot and $s \in \mathbb{R}^{17}$ for the 3 DoF robot. Finally, the state space was scaled between -1 and 1:

$$s_{scaled} = \frac{2(s - s_{low})}{s_{high} - s_{low}} - 1$$

where $s_{low} \leq s \forall s$ and $s_{high} \geq s \forall s$.

4.1.2. Action space

Two different approaches were used for modeling the action space.

Task space control: A high-level control setting where the agent controls the end-effector's position. This is achievable using the inverse kynematics of the robot to find a configuration of the robot that places the end-effector in the wanted position.

$$\begin{aligned}\bar{q}_t &= IK(\bar{p}_{ee,t}) \\ \dot{\bar{q}}_t &= \frac{\bar{q}_t - q_{t-1}}{\delta t}\end{aligned}$$

where $\bar{q}_t$ is the desired joint configuration computed using inverse kynematics from the desired end-effector's potision $p_{ee,t}$ at time t and $\dot{\bar{q}}_t$ is the desired joint velocity computed using finite differences.

The computed joint velocities were next optimized using *weighted null space optimization*, defined in 2.1, that takes into account the joint velocity limits.

Joint space control: The agent directly controls the joint accelerations $\ddot{q}$. This control variable is then converted to joint position and velocities using ATACOM [11] and submitted as a reference variable to the underlying controller. The ATACOM transformation produces a command [joint_positions, joint_velocities] that reduces greatly the number of violated constraints excluding the joint velocity constraint. Reducing the joint acceleration limits solved the joint velocity constraints problem. The action space too was scaled between -1 and 1.

4.1.3. Reward function

The designed reward is task-dependent, given that a policy is trained for each task. It is also shaped as the success reward of each task is very sparse. Generally the reward function introduces a penalty for each violated constraint proportional to its penalty points:

$$r(s, a) = r_t(s, a) + r_c(s, a), \quad (4.1)$$

where $r_t(s, a)$ is the task-dependent reward and $r_c(s, a)$ is the penalty for violating the constraints.

$$r_c(s, a) = \frac{\sum_i \left(\phi_i \cdot \frac{1}{N} \sum_{j=1}^N \sigma_j \right)}{\sum_i \phi_i}, \quad (4.2)$$

where N is the number of violated constraints, ϕ_i is the penalty points associated to the violation of constraint i (described in section 3.2.4) and σ_i is the *slack*, i.e. the difference between the current value of the constraint variable and its limit.

Additional tasks other than the ones specified in 3.1 were designed for learning behaviors useful for a complete game. A *return home* task in which the robot needs to achieve a certain configuration and remain still, and a *counter-attack* similar to the *defend* task but gives the robot the possibility to score a goal while defending from the incoming puck.

The reward function $r_t(s, a)$ for each task is now described:

Defend and counter-attack: Two agents for the defend and counter-attack tasks. In these tasks the episode starts with the puck moving towards our goal. In both cases, the agent receives a high penalty (-100) when the puck breaches our goal. When the agent successfully interacts with the puck, a reward (+10) is granted, coupled with reward proportional to the norm of the puck's velocity, with a negative sign while defending and positive sign while counter-attacking. This incentivizes the agent to interact with the puck and lower its velocity while also protecting the goal in the defend task, while pushing back the puck in the counter-attack. Moreover, in the counter-attack, the agent can also score in the opponent's goal, for which he receives an additional large bonus of 1000.

$$r = \begin{cases} 1000 & \text{if scored} \\ 10 + A \cdot \|\dot{p}_{puck}\| & \text{if interacted with puck} \\ -100 & \text{if opponent scored a goal} \end{cases}$$

where A is a constant term with positive sign in the counter-attack task, negative in the defend task.

Hit: The reward function of the hit task is based on the construction of a triangle with the opponent's area and the puck as vertices (Figure 4.1). If the puck, after the hit, lies inside the triangle, a reward is assigned, proportional to the puck's velocity; if the puck is outside the triangle, a penalty is assigned. The more the puck is outside the polygon,

the bigger the penalty.

$$r_{\text{before hit}} = \begin{cases} -\frac{\|p_{ee} - p_{puck}\|}{0.5 \text{ table_diag}} & \text{if no hit,} \\ 100 + 10 \frac{\left(1 - \left(2\frac{\alpha}{\pi}\right)^2\right) \|\dot{p}_{ee}\|}{\text{max_vel}} & \text{if hit the puck,} \\ \frac{1}{1 - \gamma} & \text{if scored} \end{cases}$$

where table_diag is the diagonal of the table and max_vel is a constant equal to the maximum velocity observable in the environment, while $\alpha = \arctan2(p_{ee,y}, p_{ee,x})$.

If the agent hits the puck, it receives an instantaneous reward and, after that, the triangle approach is applied.

$$r_{\text{after hit}} = \begin{cases} 10 + \|\dot{p}_{puck}\| & \text{if puck inside triangle,} \\ -\text{diff_angle} & \text{if otherwise} \end{cases}$$

$$\text{diff_angle} = \arctan2(\dot{p}_{puck,y}, \dot{p}_{puck,x}) - \text{angle_border}$$

,

where angle_border is the angle between the puck velocity vector and the closest triangle border.

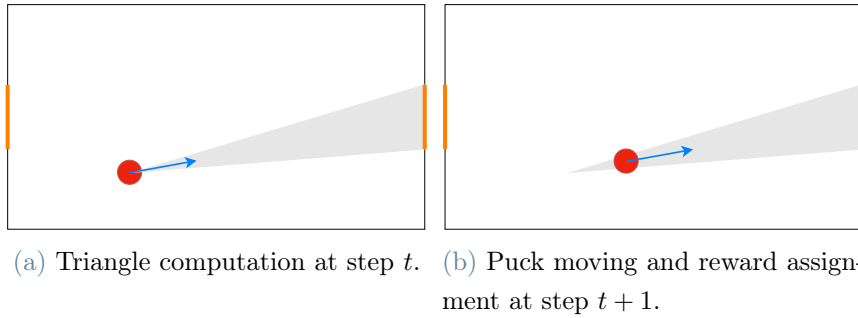


Figure 4.1: Triangle construction for assigning reward after the mallet hit the puck.

Prepare: The reward function for the prepare task is a sparse reward given at the last step with a high reward (1000) if the task is successful, otherwise -100.

Return home: At each step of the training process, the agent incurs a penalty based on

the distance to the *home configuration*

$$r = -\|q - q_{home}\|$$

where q_{home} is the initial position of the robot before a game starts or after a reset. For this task, the robot was initialized to a configuration drawn from the distribution of terminal states of the other tasks. This distribution is necessary for creating experiments with different initial positions, ensuring that the agent can learn to reach q_{home} independently of its starting position.

4.2. SAC Agent

SAC [4], described in 1.3.3 was used to implement different agents during the Robot air hockey competition. As illustrated in Figure 4.2 the agent was composed of two components: the policy’s neural network and the ATACOM transformation, described in section 2.3.

Neural Network: The trained neural network is a two layer feed forward neural network with 256 neurons per hidden layer and an output layer with $2 \times n$ neurons where n is the number of controlled joint accelerations. The input layer depends on the state space of the task’s environment. The nonlinearity of the neural network comes from the rectified linear unit (ReLU) activation function.

To achieve exploration with a continuous action space the outputs of the neural network are used as mean and standard deviation of a gaussian distribution as seen in (1.8). Finally a *tanh* transformation is applied to the action sampled from the gaussian distribution. This is done to ensure that the action is bounded between -1 and 1. The final action is interpreted as the desired joint accelerations $\ddot{q}$.

ATACOM Transformation: This component receives as inputs the desired joint accelerations $\ddot{q}$ computed from the preceding component and outputs joint positions and velocities $[q \dot{q}]$ that don’t break constraints using the ATACOM algorithm [11] described in section 2.3. However, ATACOM was unable to adhere to the constraints on the joint velocity. This is why we also limited the maximum joint accelerations between -1 and 1.

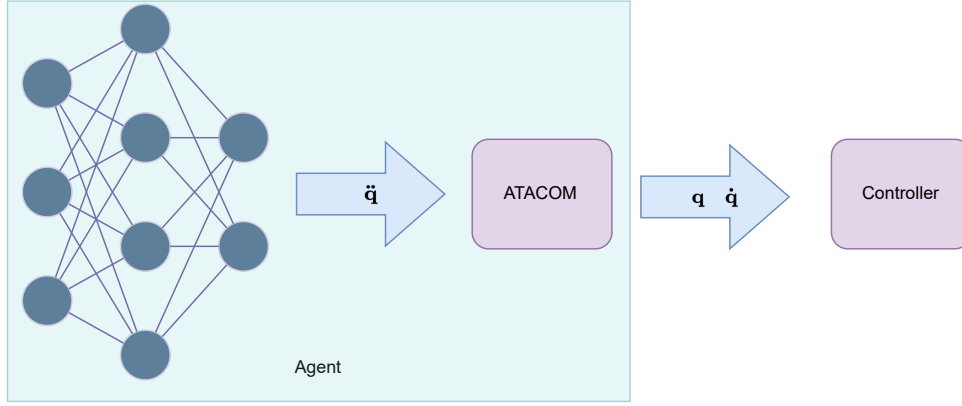


Figure 4.2: SAC Agent with ATACOM.

The SAC Agent was used for the *defend*, *counter-attack* and *home* tasks defined in section with their reward functions.

Defend and Counter-attack tasks

For the *defend* and *counter-attack* task a curriculum learning (CL) [14] approach was used to gradually increase the difficulty of the task and facilitate training. Manual curricula across three levels of complexity, easy, medium and hard, were devised. These levels differ in the initial configuration of the puck, considering its position and velocity.

1. **Easy:** In the easy level, the puck's velocity was constrained along the y-axis and kept the velocity in the x-axis low. Additionally, we positioned the puck at the end of the table, allowing the agent more reaction time.
2. **Hard:** Both the x-axis and y-axis puck's velocities were increased, and its initial position was set in the middle of the table.
3. **Medium:** The medium task served as an intermediary between the hard and easy tasks.

The agent was trained for a total of 8000 epochs, where each epoch consists in 30 episodes of experience. In the first 3000 epochs the agent was trained with the easy level. Next a *soft* switch to the *medium* level task for 1000 epochs increasing the proportion of episodes drawn from the *medium* level task. Between epoch 4000 and 6000 the agent is trained exclusively on the *medium* level task. As before, a *soft* switch to the *hard* task is performed and finally train exclusively on the *hard* level task for the last 1000 epochs.

Home task

In this additional task the goal is to train an agent able to return to the home position from any arbitrary position. The initial joint positions for the robot were drawn from a previously collected dataset of configurations. In this task the observation space was reduced to only include the joint positions and velocities:

$$s = \begin{bmatrix} q & \dot{q} \end{bmatrix}$$

The reward function is described in 4.1.3. No curriculum was needed as the *home* task is simple.

4.3. RL Optimized Rule-based Agent

For the *hit* and *prepare* task defined in 4.1.3 a rule-based controller was implemented. This controller was then optimized through PGPE [15] following the approach in [9].

In this case the robot is controlled using the *task space*, combining inverse kinematics and Anchored Quadratic Programming (AQP) [10].

The policy set was divided into 2 main branches that resemble the challenge tasks: *hit* and *prepare*. Each policy is further divided into phases, common to both of them:

- *Adjustment*: move the end-effector on (i) the line linking the puck and the goal for the *hit*; (ii) the vertical line parallel to the short side, passing through the puck, for the *prepare*.
- *Acceleration*: accelerate the end-effector and hit the puck. The *hit* policy hits the puck to score a goal, while the *prepare* one makes a more soft bump, in order to re-adjust the puck's position;
- *Final*: this phase only applies to the *hit* policy. Here, the agent keeps hitting the puck making it reach a desired acceleration, then slowly stops the end-effector following a curved trajectory.

In particular, the *hit* policy computes two quantities, $d\beta$ and ds in the following way:

$$d\beta = \begin{cases} (\theta_0 + \theta_1 \cdot t_{phase} \cdot dt) \cdot correction & \text{adjustment phase} \\ \frac{correction}{2} & \text{acceleration phase} \\ (\theta_0 + \theta_1 \cdot dt) \cdot correction & \text{slow-down phase} \end{cases}$$

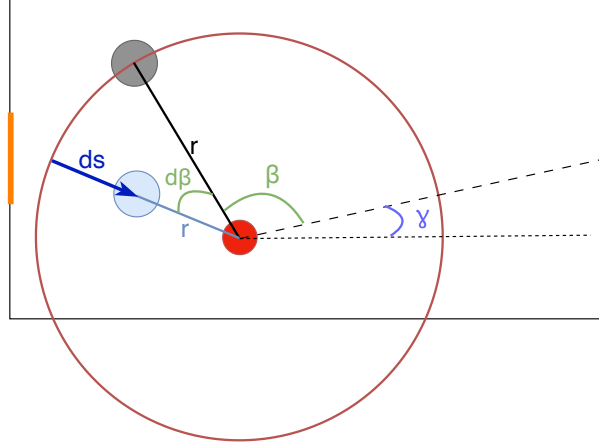


Figure 4.3: Polar coordinates used in the Rule-based controllers

$$ds = \begin{cases} \theta_2 & \text{adjustment phase} \\ \frac{ds_{t-1} + \theta_3 \cdot t_{phase} \cdot dt}{radius + r_{mallet}} & \text{acceleration phase} \\ constant & \text{slow-down phase} \end{cases}$$

where *radius* is the distance between the center of the puck and the end-effector, while *correction* is always defined as:

$$correction = \begin{cases} \beta - 90 & y_{puck} \leq 0 \\ 270 - \beta & y_{puck} > 0 \end{cases}$$

All the quantities appearing in the computation of $d\beta$ and ds are explained graphically in Figure 4.3

4.4. Hierarchical Agent

For the tournament stage of the competition, an agent capable of playing entire matches is needed. Having previously implemented different agents for each task we can play an entire match alternating between these specific agents. The difficulty now is to define how to switch between these subpolicies while being competitive against an opponent.

To do so, a high-level agent has been developed. This agent decides what policy to execute, based on the observation of the environment and on two main components: the Switcher and the Finite State Machine.

The switcher selects a new policy once the current one has met a termination condition

and informed the high level agent. This new selected policy is selected according to a set of rules applied on the observation state. The switcher’s logic is illustrated in Figure 4.4. The selected policy could potentially be the previous one. This can happen, for example, if the agent performed a prepare but the puck is still in an unsuitable position for hitting, a second prepare might be necessary.

The switcher’s rules relies on some parameters, therefore it could be trained formulating an optimization problem to find an optimal set of parameters. To do this we can use PGPE [15].

Policy Gradients with Parameter-Based Exploration (PGPE) is a reinforcement learning algorithm that enhances exploration by directly perturbing the parameters of the policy. Instead of adding noise to the actions at each step, PGPE samples the policy parameters from a distribution, leading to a deterministic execution of the policy based on these sampled parameters. This method is particularly advantageous for optimizing non-differentiable policies, such as the switcher implemented in this work. Using PGPE, a gaussian distribution ρ over the switcher’s parameters θ . The algorithm updates the mean μ and standard deviation σ of ρ , allowing effective exploration and optimization.

For further details refer to [15].

4.5. Noise Filtering

During the challenge our agents were evaluated on a remote simulator that was slightly different from the local environments we used. In order to mimic a real-world scenario this remote simulator introduced disturbances that could create problems to our agents.

The noise on the puck observations and the loss of tracking were particularly problematic. To solve this problem a kalman filter [20] was used on the puck’s position to filter the noise. Additionally, the noisy observations of joint positions and velocities were ignored and the reference joint positions and velocities given to the underlying controller at the previous step were used as observations instead.

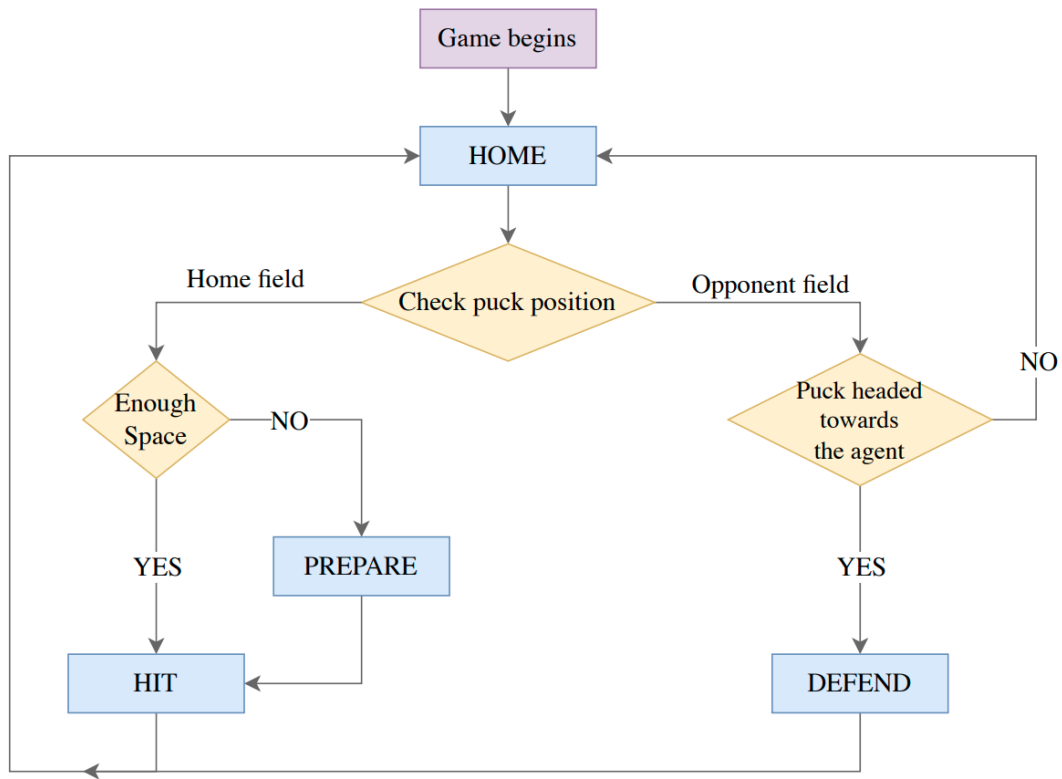


Figure 4.4: Switcher structure.

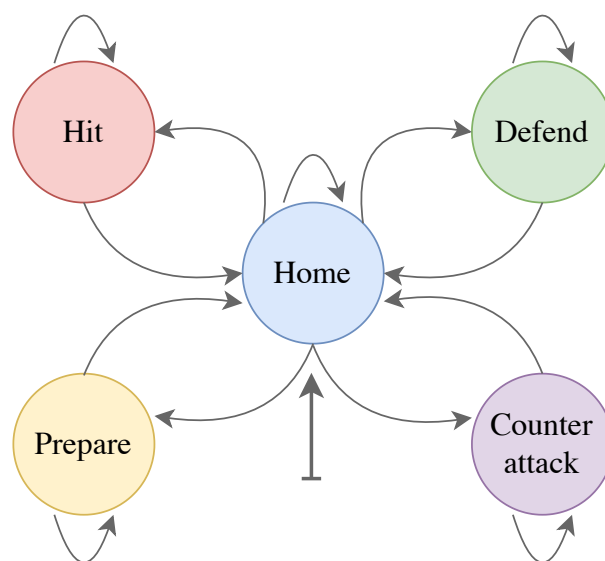


Figure 4.5: The Finite State Machine.

5 | Experimental results

This chapter describes the results of the experiments performed during the various phases of the Robot air hockey challenge and some additional improvements obtained in the following months.

5.1. Challenge Results

5.1.1. Warm up phase

In the Warm up phase only the *hit* and *defend* agents were evaluated. At that time these two agents were trained with SAC using a *Task space control* strategy. The inverse kinematics approach was employed to compute the joint positions from the desired end-effector position.

The results obtained were good in term of success rate but the agent was classified as *non deployable* as the penalty points derived from constraint violations were too high (> 1500):

Hit success	Defend success	Max Penalty points
85.0%	12.2%	4402.0

Table 5.1: Warm up 3DoF planar robot, server evaluation

5.1.2. Qualifying phase

During the qualifying phase the participants were given access to the environment with the 7 DoF robot and an additional task, the *prepare* task. In this environment, the tasks became much harder to solve while adhering to constraints as the end-effector was now capable to move in all directions given the high number of DoF. During this phase the *hit* and *defend* agent were implemented using a Rule-Based controller similar to the one described in 4.3 with a sparser reward function. Still, the hit task was the most

problematic in terms of constraints, and this contributed to having our agents classified as *improvable*.

The Defend agent was implemented using SAC as described in 4.2 using a *Joint space control* strategy coupled with ATACOM. ATACOM proved to work very well in this setting achieving a very low number of constraint violations.

The success rates and penalty points are illustrated in tables 5.2 and 5.3.

Hit success	Defend success	Prepare success	Max Penalty points
22.7 %	61.6 %	36.2 %	920.0

Table 5.2: Qualifying success 7DoF robot, server evaluation

Hit penalty	Defend penalty	Prepare penalty
920.0	1.5	329.0

Table 5.3: Qualifying penalty points

5.1.3. Tournament

During the tournament phase an agent capable of playing entire matches was needed. Adhering to constraints was also crucial because violating too many constraints would result in losing the match regardless of the score.

The employed subpolicies are the ones described in 4: a Rule-based agent was employed for hitting and preparing the puck while a SAC agent was employed to defend the goal hitting the incoming puck attempting to score the opponent’s goal. This counterattack policy was used because of the low performance of the rule-based hit policy in the hitting task. It was trained on a modified *defend environment* where the reward function includes a positive reward if the agent scores a goal as described in section 4.2.

The home policy was essential because switching policy too fast without first returning in the default position and low velocities resulted in a lot of violations. This also gave the counterattacking SAC policy an initial position it is familiar with.

The switcher policy therefore included four policies:

- hit rule-based

- counterattack SAC
- prepare rule-based
- return home SAC

For the competition, the parameters of the switcher were tuned manually. During the final phase of the competition our hierarchical agent achieved a winning rate of 50 % against the other participants always adhering to constraints.

5.2. Improving the challenge's solution

After the challenge, efforts were made to improve the solution. Specifically the objective was to have an agent capable of achieving a good success rate on the hit task and then optimize the switcher to find an optimal set of parameters.

5.2.1. Improving the Hit Task

To improve the Hit Task a SAC agent with ATACOM was used. However, our implementation of ATACOM had problems respecting the joint velocity constraints. These violations were previously indirectly solved limiting the joint accelerations given to ATACOM between -1 and 1.

First, the last joint acceleration was removed from the action space, as this joint was only responsible for the rotation of the end-effector. From now on, the acceleration given to the last joint is assumed to be 0.

Next, with trial and error a new set of joint acceleration limits was found that kept the joint velocity constraints low:

$$|\ddot{q}_{limits}| = \begin{bmatrix} 6 & 11 & 5 & 6 & 1 & 9 \end{bmatrix}$$

The previous joint acceleration limits $-1 \leq \ddot{q}_i \leq 1$ were too restrictive, while this new set of joint accelerations limits enabled our agent to be more reactive.

The reward function used for this hit task is:

$$r = \begin{cases} \frac{\|p_{puck} - p_{ee}\|}{0.5 \text{ table_diag}} & \text{if no hit} \\ 50 \cdot \|\dot{p}_{puck}\| & \text{once after hit} \\ 1000 & \text{if success} \end{cases} \quad (5.1)$$

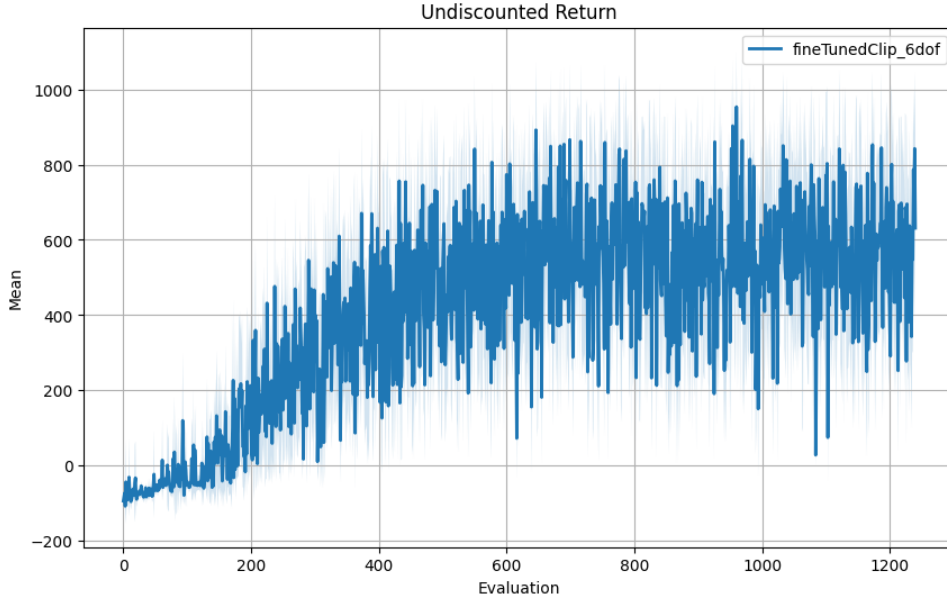


Figure 5.1: This figure represents the undiscounted mean return computed using the target policy of the agent. Each evaluation is performed every 20000 environment steps over 20 episodes.

The training environment was modified in order to aggregate all the remaining steps from the moment the puck is hit into one. Specifically, after the puck is hit, a default policy is used to make the agent stop smoothly and the training environment submits to the training algorithm only the last step of the episode. This sped up the training as all the steps during which the puck was not controllable by the agent were ignored.

A plot of the undiscounted return is illustrated in Figure 5.1. In this figure each point corresponds to the undiscounted return computed from an evaluation of the target policy over 20 episodes every 20000 environment steps. The mean return reaches a value around 600. This value is not directly linked to the success rate as the reward function introduces also a positive reward proportional to the puck’s velocity.

This new hit agent achieved on a local evaluation without noise a 58 % success rate over 100 episodes with a mean undiscounted return of 652, demonstrating an improvement over the previous rule-based hit policy.

5.2.2. Improving the Switcher

Other than replacing the rule based hit policy, a new home policy was implemented based on a PD controller:

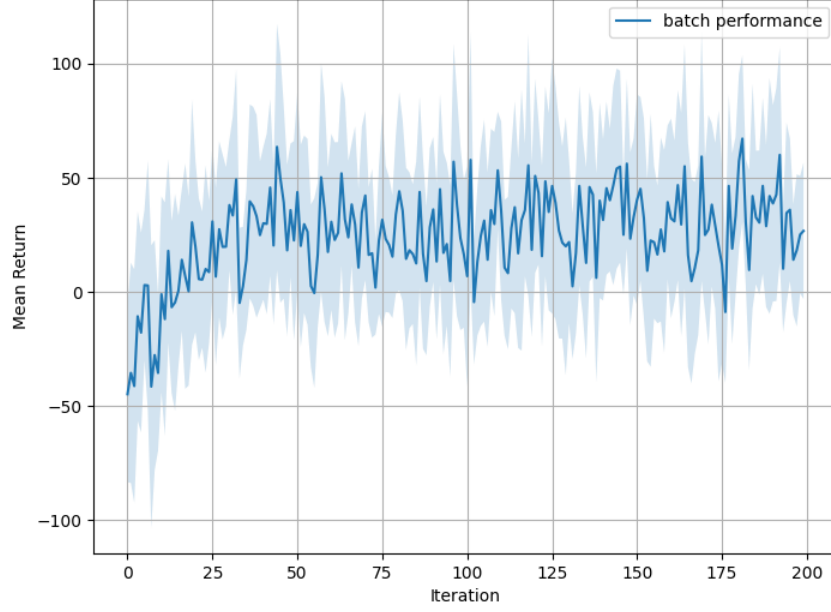


Figure 5.2: In this figure each point correspond to the mean return obtained from a distribution ρ of parameters. The return during training increases over zero, meaning that the hierarchical agent is consistently winning

$$\ddot{q} = K_p \cdot (q_{home} - q) - K_d \cdot (\dot{q})$$

with q_{home} equal to the desired position, to return to and $K_p = 10$ and $K_d = 1$. These values were found by hand.

The switcher is a non differentiable policy that depends on a parameter vector θ . It can be optimized with PGPE [15] and find an optimal set of parameters. The reward function is a simple sparse reward:

$$r = \begin{cases} 1000 & \text{if our agent scores a goal} \\ 333 & \text{if the opponent gets a fault} \\ -1000 & \text{if the opponent scores a goal} \\ -333 & \text{if our agent gets a fault} \end{cases}$$

In this training environment episodes last 3000 steps and the discount rate γ is set to 0.999. The opponent used in this environment is still the baseline provided by the competition's organizers. A plot of the mean return of the parameters distribution is illustrated in Figure 5.2.

Finally, the results of an evaluation over 100 games of 45000 steps each are illustrated in table 5.4. The mean constraint violations during the evaluation are illustrated in table 5.5. This results prove that the agent is able to outperform the baseline while keeping the constraints violations relatively low classifying it as *deployable* according section 3.2.4.

Games won	Games drawn	Games lost
65	8	27

Table 5.4: Results of the final evaluation over 100 games against the baseline opponent.

Joint velocity constraints	13.13
Joint position constraints	1.51
End-effector constraints	4.16
Link constraints constraints	1.76
Computation time	1.04

Table 5.5: Mean constraint violations during the final evaluation over 100 games against the baseline opponent.

6 | Conclusions and future developments

This thesis explored the development and implementation of a hierarchical reinforcement learning agent designed to compete in the air hockey competition, which was initiated to address the reality gap problem in reinforcement learning. The competition served as a platform to test and refine approaches that bridge the gap between simulated environments and real-world applications, ensuring that the solutions developed are robust and effective in practical scenarios.

The hierarchical agent developed in this work consists of specialized lower-level policies, each tailored to handle specific sub-tasks within the game of air hockey. These sub-policies were integrated under a rule-based high-level policy, responsible for selecting the most appropriate specialized agent during gameplay. This high-level policy was trained using PGPE, enabling effective exploration and optimization despite the non-differentiable nature of the policy.

Additionally, the ATACOM algorithm was employed to ensure that the agent operated within the defined constraints.

The results demonstrated that the hierarchical structure allowed the agent to perform effectively in the competition, showcasing improved adaptability and performance.

6.1. Future works

The developed rule-based switcher currently uses a few simple parametric rules. Future work could involve implementing a more complex switcher with additional rules that also take the opponent's actions into account. This enhancement would allow the agent to make more informed decisions based on the dynamic behavior of its opponent, leading to improved performance and adaptability in competitive scenarios.

Another strategy could be to apply an end-to-end hierarchical reinforcement learning

approach, such as HIRO [13]. In this approach, a single low-level policy receives a latent space vector from the high-level policy, determining the desired behavior. This method could streamline the decision-making process and potentially yield more efficient and effective learning, as the high-level policy provides structured guidance to the low-level policy.

Given that air hockey is inherently a multi-agent setting, investigating a self-play learning strategy could also be beneficial. Self-play allows the agent to improve by continuously adapting to and learning from interactions with itself or other agents. This approach can foster the development of robust strategies, as the agent is exposed to a wide range of behaviors and tactics, leading to a more comprehensive understanding of the game dynamics.

Bibliography

- [1] Air hockey challenge '23. URL <https://air-hockey-challenge.robot-learning.net/>.
- [2] C. D'Eramo, D. Tateo, A. Bonarini, M. Restelli, and J. Peters. Mushroomrl: Simplifying reinforcement learning research. *Journal of Machine Learning Research*, 22(131):1–5, 2021. URL <http://jmlr.org/papers/v22/18-056.html>.
- [3] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods, 2018.
- [4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *CoRR*, abs/1801.01290, 2018. URL <http://arxiv.org/abs/1801.01290>.
- [5] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. *CoRR*, abs/1812.05905, 2018. URL <http://arxiv.org/abs/1812.05905>.
- [6] H. Hasselt. Double q-learning. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010. URL https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fced301b442654dd8c23b3fc9-Paper.pdf.
- [7] J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *International Journal of Robotics Research*, 32(11):1238–1274, 2013. doi: 10.1177/0278364913495721.
- [8] A. D. Laud. *Theory and application of reward shaping in reinforcement learning*. PhD thesis, USA, 2004. AAI3130966.
- [9] A. Likmeta, A. M. Metelli, A. Tirinzoni, R. Giol, M. Restelli, and D. Romano. Combining reinforcement learning with rule-based controllers for transparent and general decision-making in autonomous driving. *Robotics and Au-*

- onomous Systems*, 131:103568, 2020. ISSN 0921-8890. doi: <https://doi.org/10.1016/j.robot.2020.103568>. URL <https://www.sciencedirect.com/science/article/pii/S0921889020304085>.
- [10] P. Liu, D. Tateo, H. Bou-Ammar, and J. Peters. Efficient and reactive planning for high speed robot air hockey. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 586–593, 2021. doi: 10.1109/IROS51168.2021.9636263.
 - [11] P. Liu, D. Tateo, H. Bou-Ammar, and J. Peters. Robot reinforcement learning on the constraint manifold. In A. Faust, D. Hsu, and G. Neumann, editors, *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 1357–1366. PMLR, 2022.
 - [12] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller. Playing atari with deep reinforcement learning. *CoRR*, abs/1312.5602, 2013. URL <http://arxiv.org/abs/1312.5602>.
 - [13] O. Nachum, S. Gu, H. Lee, and S. Levine. Data-efficient hierarchical reinforcement learning, 2018.
 - [14] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone. Curriculum learning for reinforcement learning domains: A framework and survey, 2020. URL <https://arxiv.org/abs/2003.04960>.
 - [15] F. Sehnke, C. Osendorfer, T. Rückstieß, A. Graves, J. Peters, and J. Schmidhuber. Parameter-exploring policy gradients. *Neural Networks*, 23(4):551–559, 2010. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2009.12.004>. URL <https://www.sciencedirect.com/science/article/pii/S0893608009003220>. The 18th International Conference on Artificial Neural Networks, ICANN 2008.
 - [16] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.
 - [17] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.
 - [18] N. Vahrenkamp, T. Asfour, G. Metta, G. Sandini, and R. Dillmann. Manipulability analysis. In *2012 12th IEEE-RAS International Conference on Humanoid Robots (Humanoids 2012)*, pages 568–573, 2012. doi: 10.1109/HUMANOIDS.2012.6651576.
 - [19] C. J. C. H. Watkins. *Learning from Delayed Rewards*. PhD thesis, King’s Col-

lege, Cambridge, UK, May 1989. URL http://www.cs.rhul.ac.uk/~chrisw/new_thesis.pdf.

[20] G. Welch, G. Bishop, et al. An introduction to the kalman filter. 1995.

List of Figures

2.1	Ball Paddling Robot	25
2.2	ATACOM	29
3.1	Planar robot base frame.	31
3.2	KUKA robot base frame.	31
3.3	Challenge framework.	36
3.4	Control loop.	37
3.5	The air hockey table.	38
4.1	Triangle construction for assigning reward after the mallet hit the puck. . .	46
4.2	SAC Agent with ATACOM.	48
4.3	Polar coordinates used in the Rule-based controllers	50
4.4	Switcher structure.	52
4.5	The Finite State Machine.	52
5.1	Undiscounted return.	56
5.2	PGPE mean return.	57

List of Tables

3.1	3 DoF planar robot specifications.	38
3.2	7 DoF robot specifications.	39
5.1	Warm up 3DoF planar robot, server evaluation	53
5.2	Qualifying success 7DoF robot, server evaluation	54
5.3	Qualifying penalty points	54
5.4	Results of the final evaluation.	58
5.5	Results of the final evaluation - Mean constraint violations.	58

List of Acronyms

Acronym	Description
<i>MDP</i>	Markov Decision Process
<i>RL</i>	Reinforcement Learning
<i>SAC</i>	Soft Actor Critic
<i>DoF</i>	Degrees of Freedom
<i>PGPE</i>	Policy Gradient with Parameter-based Exploration
<i>FK</i>	Forward Kynematics
<i>IK</i>	Inverse Kynematics
<i>QP</i>	Quadratic Programming
<i>AQP</i>	Anchored Quadratic Programming

